

Projeto de Sistemas

Gustavo Campos Menezes

Edson Marchetti da Silva

Curso Técnico em Planejamento e
Gestão em Tecnologia da Informação





·rede
e-Tec
Brasil

Projeto de Sistemas

Gustavo Campos Menezes

Edson Marchetti da Silva



CEFET-MG

Belo Horizonte - MG
2013

Presidência da República Federativa do Brasil
Ministério da Educação
Secretaria de Educação Profissional e Tecnológica

© Centro Federal de Educação Tecnológica de Minas Gerais
Este Caderno foi elaborado em parceria entre o Centro Federal de Educação Tecnológica de Minas Gerais e a Universidade Federal de Santa Catarina para a Rede e-Tec Brasil

Equipe de Elaboração

Centro Federal de Educação Tecnológica de Minas Gerais – CEFET-MG

Coordenação Institucional

José Wilson da Costa/CEFET-MG

Coordenação do Curso

Adelson de Paula Silva/CEFET-MG

Professores-autores

Gustavo Campos Menezes/CEFET-MG

Edson Marchetti da Silva/CEFET-MG

Comissão de Acompanhamento e Validação

Universidade Federal de Santa Catarina – UFSC

Coordenação Institucional

Araci Hack Catapan/UFSC

Coordenação do Projeto

Silvia Modesto Nassar/UFSC

Coordenação de Design Instrucional

Beatriz Helena Dal Molin/UNIOESTE e UFSC

Coordenação de Design Gráfico

Juliana Tonietto/UFSC

Design Instrucional

Renato Cislighi/UFSC

Web Master

Rafaela Lunardi Comarella/UFSC

Web Design

Beatriz Wilges/UFSC

Mônica Nassar Machuca/UFSC

Diagramação

Bárbara Zardo/UFSC

Roberto Colombo/UFSC

Revisão

Júlio César Ramos/UFSC

Projeto Gráfico

e-Tec/MEC

Catálogo na fonte pela Biblioteca Universitária da
Universidade Federal de Santa Catarina

M543p Menezes, Gustavo Campos

**Projeto de sistemas / Gustavo Campos Menezes,
Edson Marchetti da Silva. – Belo Horizonte : CEFET-MG, 2013.
88 p. : il., tabs.**

Inclui bibliografia

ISBN: 978-85-99872-25-3

**1. Projeto de sistemas. 2. Métodos orientados
a objetos. 3. UML (Computação). I. Silva, Edson
Marchetti da. Título.**

CDU: 681.31.06

Apresentação e-Tec Brasil

Bem-vindo a Rede e-Tec Brasil!

Você faz parte de uma rede nacional de ensino, que por sua vez constitui uma das ações do Pronatec - Programa Nacional de Acesso ao Ensino Técnico e Emprego. O Pronatec, instituído pela Lei nº 12.513/2011, tem como objetivo principal expandir, interiorizar e democratizar a oferta de cursos de Educação Profissional e Tecnológica (EPT) para a população brasileira propiciando caminho de o acesso mais rápido ao emprego.

É neste âmbito que as ações da Rede e-Tec Brasil promovem a parceria entre a Secretaria de Educação Profissional e Tecnológica (SETEC) e as instâncias promotoras de ensino técnico como os Institutos Federais, as Secretarias de Educação dos Estados, as Universidades, as Escolas e Colégios Tecnológicos e o Sistema S.

A educação a distância no nosso país, de dimensões continentais e grande diversidade regional e cultural, longe de distanciar, aproxima as pessoas ao garantir acesso à educação de qualidade, e promover o fortalecimento da formação de jovens moradores de regiões distantes, geograficamente ou economicamente, dos grandes centros.

A Rede e-Tec Brasil leva diversos cursos técnicos a todas as regiões do país, incentivando os estudantes a concluir o ensino médio e realizar uma formação e atualização contínuas. Os cursos são ofertados pelas instituições de educação profissional e o atendimento ao estudante é realizado tanto nas sedes das instituições quanto em suas unidades remotas, os polos.

Os parceiros da Rede e-Tec Brasil acreditam em uma educação profissional qualificada – integradora do ensino médio e educação técnica, - é capaz de promover o cidadão com capacidades para produzir, mas também com autonomia diante das diferentes dimensões da realidade: cultural, social, familiar, esportiva, política e ética.

Nós acreditamos em você!

Desejamos sucesso na sua formação profissional!

Ministério da Educação
Janeiro de 2013

Nosso contato
etecbrasil@mec.gov.br

Indicação de ícones

Os ícones são elementos gráficos utilizados para ampliar as formas de linguagem e facilitar a organização e a leitura hipertextual.



Atenção: indica pontos de maior relevância no texto.



Saiba mais: oferece novas informações que enriquecem o assunto ou “curiosidades” e notícias recentes relacionadas ao tema estudado.



Glossário: indica a definição de um termo, palavra ou expressão utilizada no texto.



Mídias integradas: sempre que se desejar que os estudantes desenvolvam atividades empregando diferentes mídias: vídeos, filmes, jornais, ambiente AVEA e outras.



Atividades de aprendizagem: apresenta atividades em diferentes níveis de aprendizagem para que o estudante possa realizá-las e conferir o seu domínio do tema estudado.

Sumário

Palavra do professor-autor	9
Apresentação da disciplina	11
Projeto instrucional	13
Aula 1 – Teoria de sistemas	15
1.1 Constituição de sistemas	15
1.2 Descrição dos sistemas	16
1.3 Por que projeto de sistemas?	17
1.4 Problemas enfrentados	17
1.5 O perfil que um analista de sistemas deve possuir	18
1.6 Características que um sistema deve possuir	19
1.7 Metodologias de desenvolvimento de <i>software</i>	19
1.8 Ciclos de vida do desenvolvimento de sistemas	20
1.9 Modelos de desenvolvimento	20
Aula 2 – Levantamento de dados e análise essencial	25
2.1 Entrevistas	25
2.2 Reunião	27
2.3 <i>Joint Application Design (JAD)</i>	29
2.4 Análise essencial	31
2.5 Estrutura da análise essencial	32
2.6 Modelo ambiental	33
2.7 Modelo comportamental	34
2.8 Lista de eventos	35
Aula 3 – Análise orientada a objetos – introdução à UML	41
3.1 Características	41
3.2 Fundamentos da orientação a objetos	42
3.3 Conceitos e princípios da OO	42

Aula 4 – <i>Unified Modeling Language</i> (UML)	47
4.1 O que é a UML?	47
4.2 Estrutura da UML	48
4.3 Diagrama de atividade	49
4.4 Diagrama de caso de uso	49
4.5 Diagrama de transição de estado	50
4.6 Diagrama de sequência	51
4.7 Diagrama de colaboração ou comunicação	52
4.8 Diagrama de classe	52
4.9 Diagrama de componente	52
4.10 Diagrama de objeto	53
4.11 Diagrama de instalação	54
4.12 Diagrama de pacote	54
Aula 5 – Diagrama de caso de uso	57
5.1 O que é diagrama de caso de uso?	57
5.2 Maneira de documentar um caso de uso	57
5.3 Exemplo de um caso de uso	58
5.4 Representação gráfica de um caso de uso	59
5.5 Conclusões sobre caso de uso	61
5.6 Exemplos e exercícios	62
5.7 Diagrama de classes	64
5.8 Relacionamentos entre classes	65
5.9 Exemplo de um diagrama de classe	70
Aula 6 – Demais diagramas da UML	73
6.1 Diagrama de atividade	73
6.2 Diagrama de sequência	75
6.3 Diagrama de transição de estado	78
6.4 Diagrama de pacotes	80
6.5 Diagrama de componentes e implantação	81
6.6 Diagrama de implantação	83
Referências	86
Currículo dos professores-autores	87

Palavra do professor-autor

Prezado estudante!

É um prazer tê-lo conosco!

O CEFET-MG oferece a você, juntamente com as prefeituras e o Governo Federal, o Curso Técnico em Planejamento e Gestão em Tecnologia da Informação (PGTI). Embora o curso seja oferecido na modalidade a distância, espero que haja uma grande interação entre nós. Atualmente, com os recursos na área de Tecnologia da Informação (*e-mail*, *chat*, videoconferência etc.), podemos manter uma comunicação bastante produtiva.

O conteúdo de Projeto de Sistemas é, juntamente com as disciplinas de Programação de Computadores, fundamental para a formação de um Técnico em Planejamento e Gestão em Tecnologia da Informação. Enquanto nas disciplinas de Linguagens de Programação você desenvolveu as habilidades de raciocínio e lógica de programação, em Projeto de Sistemas você irá aprender a projetar sistemas. Ou seja, irá conhecer as metodologias de desenvolvimento de sistemas, as técnicas para levantamento de dados e ferramentas para modelagem de sistemas.

Nunca se esqueça de que em qualquer curso, seja ele presencial ou a distância, o principal responsável pelo sucesso é você.

Então, desejamos a você SUCESSO!!!

Professor Gustavo Campos Menezes
e Professor Edson Marchetti da Silva

Apresentação da disciplina

Prezado estudante!

Produzimos este caderno com o objetivo de servir como um guia básico de referência para a disciplina de Projeto de Sistemas. Nesta disciplina vamos estudar os principais conceitos relativos ao processo de desenvolvimento de um sistema, seja ele um aplicativo para um *desktop*, uma página da internet ou um grande sistema integrado para uma organização.

Além disso, no final do caderno é apresentada a bibliografia complementar que pode ajudar na compreensão de cada conteúdo abordado.

Bons estudos!

Projeto instrucional

Disciplina: Projeto de Sistemas (carga horária: 60 horas).

Ementa: Conceitos básicos sobre Engenharia de *software* e análise de requisitos. Técnicas de levantamento de dados. Análise essencial. Modelagem de sistemas com UML.

AULA	OBJETIVOS DE APRENDIZAGEM	MATERIAIS	CARGA HORÁRIA (horas)
1. Teoria de sistemas	Conhecer os conceitos fundamentais sobre sistemas e sobre o ciclo de vida do desenvolvimento de <i>software</i> .	Fórum de discussões, <i>chat</i> .	10
2. Levantamento de dados e análise essencial	Conhecer as principais técnicas de levantamento de dados existentes bem como as ferramentas da análise essencial de sistemas.	Fórum de discussões, <i>chat</i> , <i>softwares</i> de diagrama.	10
3. Análise orientada a objetos – Introdução à UML	Estudar os principais conceitos referentes à orientação a objetos. Utilizar alguns dos principais diagramas da UML para modelagem de sistemas.	Fórum de discussões, <i>chat</i> , exemplos de diagramas e código.	10
4. UML (<i>Unified Modeling Language</i>)	Dominar os principais diagramas da UML para o desenvolvimento de <i>software</i> .	Fórum de discussões, <i>chat</i> , exemplos de diagramas.	10
5. Diagrama de caso de uso	Analisar e criar diagrama de casos de uso para o seu sistema. Analisar e criar diagramas de classe.	Fórum de discussões, <i>chat</i> , exemplos de diagramas e estudos de caso.	10
6. Demais diagramas da UML	Criar e analisar os diagramas apresentados.	Fórum de discussões, <i>chat</i> , exemplos de diagramas e exercícios.	10

Aula 1 – Teoria de sistemas

Objetivos

Entender o que é um sistema.

Conhecer os processos de desenvolvimento de sistemas existentes.

A **Teoria de Sistemas** (TS) é um ramo específico da Teoria Geral de Sistemas (TGS). A Teoria Geral de Sistemas (TGS) surgiu com os trabalhos do biólogo alemão Ludwig von Bertalanffy, o qual criticava a visão que se tem do mundo dividido em diferentes áreas como Física, Química, Biologia etc. São divisões arbitrárias e com fronteiras definidas. A natureza não está dividida em nenhuma dessas partes.

A TGS fundamenta-se em três premissas básicas:

- a) os sistemas existem dentro de sistemas;
- b) os sistemas são abertos;
- c) as funções de um sistema dependem de sua estrutura, ou seja, a soma das partes é menor que o todo. **Todo = Soma das partes + Relacionamento entre partes.**

1.1 Constituição de sistemas

- **Sistemas físicos ou concretos:** quando são compostos de equipamentos, de maquinaria e de objetos ou coisas reais (*hardware*).
- **Sistemas abstratos ou conceituais:** quando compostos de conceitos, planos, hipóteses e ideias (*software*).
- **Sistemas fechados:** são os sistemas que não apresentam intercâmbio com o meio ambiente que os circunda, pois são herméticos a qualquer influência ambiental.
- **Sistemas abertos:** são os sistemas que apresentam relações de intercâmbio com o ambiente, através de entradas e saídas.

Modelo genérico de um sistema aberto:

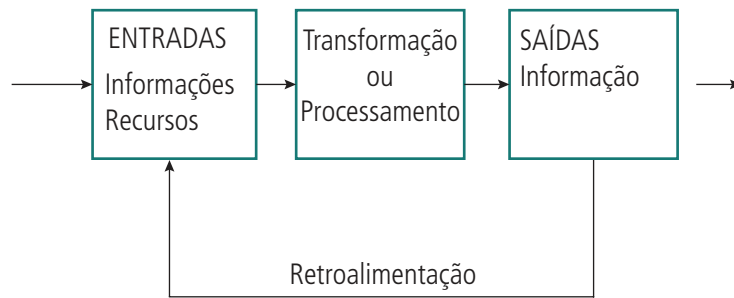


Figura 1.1: Modelo de sistema aberto

Fonte: Elaborada pelo autor Gustavo Menezes

Os sistemas abertos são compostos por seus elementos (partes) e as relações entre eles:

- a) **entrada ou insumo** – é a força de arranque de um sistema; ela permite a operação do sistema;
- b) **processamento ou transformador** – é o fenômeno que produz mudança, ou seja, converte entradas em saídas;
- c) **saída ou resultado** – é a finalidade para qual se reuniram elementos e relações do sistema. Devem ser coerentes com a finalidade do sistema;
- d) **retroação ou retroalimentação** – é a função que visa comparar a saída a determinados padrões estabelecidos. Visa manter ou aperfeiçoar o desempenho do processo;
- e) **ambiente** – é o meio que envolve externamente o sistema.

1.2 Descrição dos sistemas

Prezado estudante, nenhum sistema pode ser desenvolvido, ou seja, você NUNCA deve iniciar a programação de um sistema sem antes descrevê-lo. Para isso é necessário responder aos seguintes itens:

- **Objetivos do sistema** – O que o sistema deve fazer?
- **Ambiente** – Onde o sistema está inserido (fronteira com o meio exterior)?
- **Recursos** – Quais são os insumos retirados do ambiente para executar transformações?
- **Componentes** – Quais são os elementos essenciais para seu funcionamento?
- **Administração** – Quais são os métodos ou componentes que utilizam os recursos para atingir os objetivos?

1.3 Por que projeto de sistemas?

Alguns dos motivos que levam as organizações a desenvolverem sistemas: aumento da qualidade dos produtos e serviços; aumento da vantagem competitiva sobre os concorrentes; aumento do nível de serviço, do desempenho dos recursos humanos.

Na disciplina Projetos de Sistemas, vamos estudar métodos para desenvolver sistemas. Muitas vezes queremos programar diretamente nossos sistemas em vez de projetá-los anteriormente. Porém, essa não é uma boa prática. Estudos feitos nos EUA para medir a produtividade no desenvolvimento de sistemas (Standish Group), disponível em: <http://blog.standishgroup.com/>, revelaram que:

- **30% a 40%** dos projetos são **cancelados**;
- **50%** dos projetos custam mais que o **dobro do que foi** inicialmente projetado;
- **15 a 20%** dos projetos terminam **dentro do prazo e dentro do orçamento**.

Os principais problemas encontrados foram:

- falta de qualidade do produto final;
- não cumprimento dos prazos;
- não cumprimento dos custos.

1.4 Problemas enfrentados

Os problemas encontrados no desenvolvimento de um sistema podem ocorrer a partir de dois pontos de vista.

Na visão do usuário, ocorre que os sistemas:

- a) não fazem o que deveriam fazer;
- b) são entregues tarde demais;
- c) são caros;
- d) de baixa qualidade;
- e) com muitos defeitos;
- f) difíceis de usar, e
- g) lentos.

Já na visão do analista, este observa que os usuários:

- a) não sabem o que realmente querem;
- b) não fornecem os detalhes de forma precisa;
- c) são desinteressados;
- d) têm medo de perder o poder por deixar de reter a informação.

Independentemente da metodologia, do processo de desenvolvimento de *software*, existe sempre um “ruído” na informação. Ou seja, nem sempre o que o usuário realmente deseja é aquilo que ele consegue transmitir ao analista/programador, e nem sempre o analista/programador consegue entender claramente o que o usuário quer.

Alguns dos desafios encontrados pelos desenvolvedores de *software*

- a) Técnico-econômico:
 - produtividade da equipe;
 - confiabilidade dos programas;
 - segurança dos dados;
 - desempenho;
 - portabilidade.
- b) Psicossocial:
 - sensação de perda;
 - falta de comunicação;
 - falta de entendimento.

1.5 O perfil que um analista de sistemas deve possuir

Caro estudante, para minimizar ao máximo os transtornos causados no processo de desenvolvimento de sistemas, precisamos nos esforçar para atender a alguns requisitos. Dessa forma, no nosso trabalho precisamos constantemente buscar as seguintes habilidades:

- a) **comunicação:** ter capacidade de ouvir, redigir e expor ideias com clareza e precisão;
- b) **capacidade de análise:** ter aptidão para realizar operações mentais com abstrações;

- c) **capacidade de negociação:** saber administrar conflitos de interesse pessoais. Ser conciliador. Saber trabalhar em equipe;
- d) **conhecimento da área:** ter conhecimento pessoal adquirido por meio da vivência com as operações da empresa;
- e) **gerenciamento de projeto:** ter a capacidade de gerir a alocação dos recursos humanos e materiais diante da disponibilidade financeira e do tempo;
- f) **conhecimento técnico:** ter a capacidade de especificar sistemas de informação utilizando ferramentas de modelagem, linguagem de programação, banco de dados, arquitetura de sistemas e redes de computadores.

1.6 Características que um sistema deve possuir

Todo sistema construído, independentemente do tamanho, aplicação, complexidade, deve também atender a algumas características principais:

- **confiabilidade:** deve possuir o menor número de defeitos (“bugs”) possível;
- **desempenho:** deve possuir um tempo de resposta aceitável para o usuário;
- facilidade de uso: deve ser de fácil utilização;
- **flexibilidade:** deve ter a possibilidade de alteração/evolução de maneira fácil.

1.7 Metodologias de desenvolvimento de software

Atualmente, existem diversas “receitas” para desenvolver *software*. Cada empresa adota a metodologia que julga mais eficiente para as suas necessidades. Abaixo, seguem alguns conceitos importantes:

- a) **metodologia:** descrição sobre a maneira de utilizar um conjunto coerente e coordenado de métodos para atingir um objetivo, de modo a se evitar a subjetividade na execução do trabalho;
- b) **método:** procedimento adotado para atingir um objetivo;
- c) **técnica:** modo apropriado de investigar sistematicamente um determinado universo de interesse do problema;
- d) **notação:** conjunto de caracteres, símbolos e sinais formando um sistema convencionado de representação.

1.8 Ciclos de vida do desenvolvimento de sistemas

Um sistema pode ser interpretado como um organismo. Dessa forma, todo sistema tem o seu início, desenvolvimento e término. Todo processo de desenvolvimento apresenta todas ou algumas das etapas a seguir:

- **análise e definição de requisitos:** por meio de técnicas de entrevistas, são definidas em detalhes as principais funções que o sistema deve possuir, para a sua especificação;
- **projeto de sistemas e software:** o projeto do sistema agrupa os requisitos em *hardware* e *software*;
- **implementação e teste de unidade:** nessa fase o projeto do *software* é visto como um conjunto de programas ou unidades de programa. O teste de unidade tem como objetivo verificar se cada unidade ou módulo atende à especificação;
- **integração e teste de sistemas:** os módulos ou unidades do programa são integrados e testados como um programa completo. Depois dos testes, o programa é entregue ao cliente;
- **operação e manutenção:** fase considerada como a mais longa do ciclo de vida. O sistema é instalado e colocado em operação. À medida que novos requisitos ou defeitos são descobertos, o sistema sofre manutenções e atualizações.

1.9 Modelos de desenvolvimento

a) Codifica-remenda

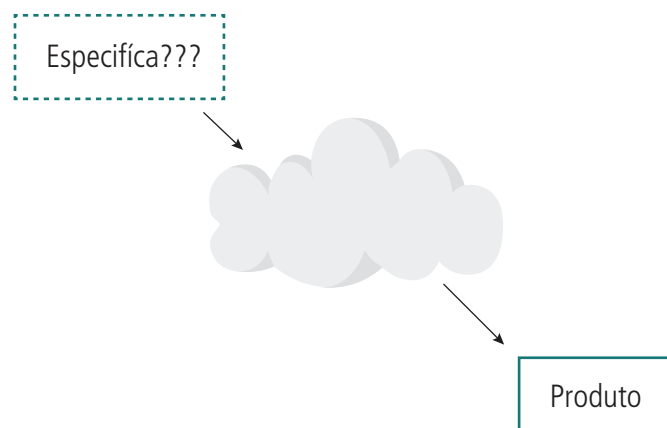


Figura 1.2: Codifica-remenda.

Fonte: Adaptada de Paula Filho (2003, p. 18)

Partindo de uma especificação, ou nem isto, os desenvolvedores começam imediatamente a codificar, remendando a medida que os erros acontecem. Possui apenas pontos negativos. Normalmente o sistema nunca termina ou é abandonado.

b) Cascata

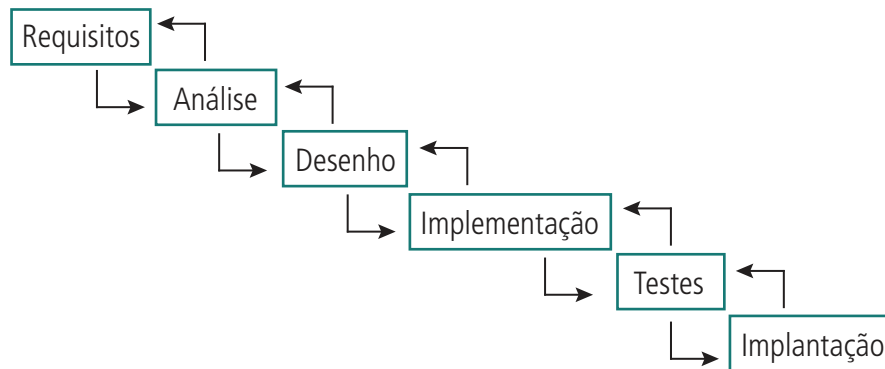


Figura 1.3: Cascata

Fonte: Adaptada de Paula Filho (2003, p. 18)

Os subprocessos são executados em estrita sequência. O que permite demarcá-los com pontos de controle bem definidos. Esse modelo possui baixa visibilidade para o cliente, que só recebe o resultado no final do projeto.

c) Espiral

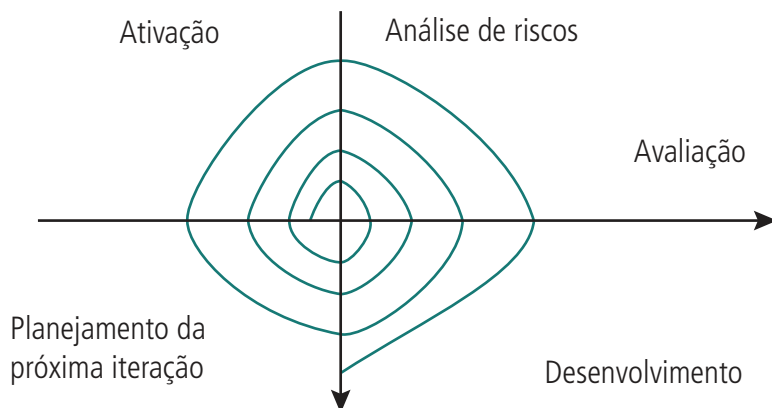


Figura 1.4: Processo em espiral

Fonte: Adaptada de Paula Filho (2003, p. 19)

Projeto é desenvolvido em uma série de iterações, em que cada iteração corresponde a uma volta na espiral. Permite construir produto funcional em prazos curtos. Difícil de gerenciar o projeto.

d) Prototipagem evolutiva

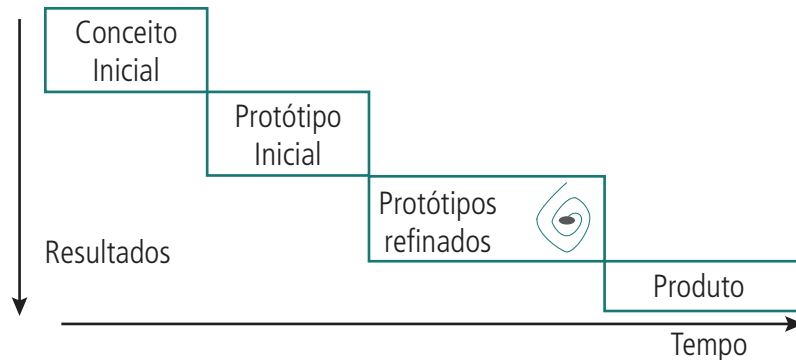


Figura 1.5: Prototipagem evolutiva

Fonte: Adaptada de Paula Filho (2003, p. 20)

A combinação dos modelos Cascata e Prototipagem evolutiva permite que em pontos bem definidos os usuários possam avaliar parte do produto.

e) Entrega evolutiva

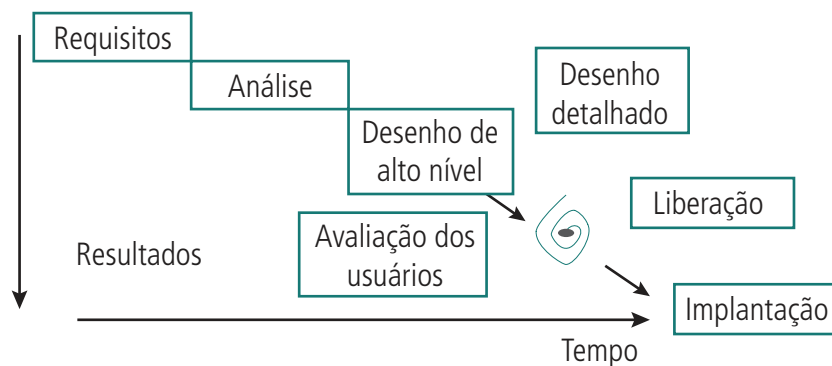


Figura 1.6: Entrega evolutiva.

Fonte: Adaptada de Paula Filho (2003, p. 20)

A espiral é usada para construir uma série de versões provisórias chamadas protótipos. Maior interação entre desenvolvedor e cliente.

Atenção: Antes de iniciarmos a próxima aula, seguem algumas dicas para o projeto de uma interface:

- a) divida a tela em regiões;
- b) agrupe os dados logicamente;
- c) alinhe os dados verticalmente;
- d) explicita o tamanho dos campos de entrada;
- e) evite telas poluídas;

- f) evite exagero no uso de recursos visuais;
- g) forneça mensagens elucidativas e objetivas;
- h) peça confirmação para funções críticas (como exclusão);
- i) proporcione feedback;
- j) ordene as listas exibidas;
- k) não exija preenchimento de caracteres não significativos;
- l) utilize um vocabulário consistente;
- m) considere a sequência dos dados do documento associado;
- n) seja consistente, mantenha padrões.

Resumo

Prezado estudante, nesta aula estudamos os principais conceitos relacionados à Teoria de Sistemas e também os processos de desenvolvimento. Além disso, vimos os principais problemas enfrentados pelo analista de sistemas durante a concepção de um *software*.

Atividades de aprendizagem

Você foi convidado para desenvolver um sistema informatizado que deverá controlar uma oficina de uma concessionária de veículos. O cliente leva o carro ao consultor técnico que o encaminha à oficina para diagnóstico e orçamento. Se aprovado, executa o serviço e troca as peças. Sabe-se que o sistema de vendas já está implantado. Deseja-se emitir a NF a partir da OS fechada.

Identifique:

- a) Quais são as principais funcionalidades?
- b) Quantos programas serão necessários desenvolver?
- c) Quem são e em que momentos os usuários irão interagir com o sistema?
- d) Quais serão as entradas e saídas de dados?
- e) Quais arquivos e campos serão necessários ser criados?

Registre suas respostas em um arquivo texto e poste-o no AVEA.

Aula 2 – Levantamento de dados e análise essencial

Objetivos

Conhecer as principais técnicas de levantamento de dados existentes.

Conhecer as ferramentas da análise essencial de sistemas.

Os problemas com que o analista de sistemas deve se deparar são na maioria das vezes extremamente complexos. Dessa forma é difícil sem o uso de alguma técnica estabelecer claramente o que o sistema deve fazer. Os requisitos de um sistema são basicamente as funções que ele deve possuir para desempenhar corretamente o seu papel dentro de uma organização. Basicamente um requisito pode ser definido como a descrição das funções e restrições para o sistema. A seguir são apresentadas algumas técnicas adotadas para estabelecer uma coleta de dados ou levantamento de requisitos:

- a) entrevistas;
- b) pesquisa em manuais ou documentos internos;
- c) reunião em grupo/*Joint Application Design* (JAD);
- d) questionários;
- e) *brainstorming*.

2.1 Entrevistas

Uma das atividades importantes do analista de sistemas é conseguir extrair as informações mais relevantes para o desenvolvimento do sistema. Para isso, uma das técnicas adotada é a entrevista. A seguir são apresentadas as principais etapas desse processo de levantamento de dados:

a) preparação prévia

Além de conhecer as técnicas para uma boa entrevista, a preparação do ambiente é também fundamental para o sucesso desse processo.

- pesquisar previamente o problema:

- Estudar a organização:
 - ambiente interno: ramo de negócio, produtos e serviços, estrutura organizacional;
 - ambiente externo: concorrência, pontos fortes e fracos
- Organizar infraestrutura:
 - desenvolver agenda de atividades e encaminhar com antecedência para a empresa;
 - indicar os responsáveis;
 - verificar se os participantes confirmaram presença;
 - requisitar salas e equipamentos;
 - no dia anterior, confirmar a reunião.

b) estruturação

Antes de partir para a execução da entrevista, fique atento aos seguintes itens:

- adequar condutas ao contexto (nível hierárquico do entrevistado);
- partir do geral para o específico;
- entender os processos: quando, quem e por quê;
- resumir oralmente a entrevista no final;
- gerar documento escrito após a entrevista (ata).

c) condução

A condução é fundamental para o sucesso; se o responsável por conduzir a entrevista não demonstrar confiança, não estabelecer uma boa conexão com as pessoas, o fracasso é inevitável. Fique atento para as seguintes dicas:

- apresente-se e justifique sua atribuição;
- seja mais informal;
- separe fatos de opiniões;

- não jogue em nenhum time;
- a responsabilidade em decidir é do usuário;
- não prolongue, seja objetivo.

d) documentação

Não se esqueça de documentar a entrevista. É comum o analista ouvir os relatos dos profissionais e só depois de uma hora de conversa começar a anotar as informações. Essa não é uma boa prática: O analista com certeza irá esquecer ou confundir alguns conceitos; o que normalmente acontece, e que causa uma imagem negativa, é o analista/entrevistador repetir algumas perguntas. Isso causa atraso, retrabalho e uma péssima imagem para o desenvolvedor do sistema.

2.2 Reunião

Durante a reunião é necessário que o responsável consiga conduzir com sucesso o trabalho e manter sempre a atenção no perfil dos participantes.

2.2.1 Conduta

Os pontos-chave a considerar são listados a seguir:

- a)** reforçar as colocações interessantes dos participantes;
- b)** ignorar comentários fora do objetivo, dar foco;
- c)** interromper diálogos paralelos;
- d)** esforçar-se para obter a opinião de todos, inclusive dos mais tímidos e subalternos;
- e)** escrever as decisões em local visível a todos os participantes;
- f)** olhar para quem está falando;
- g)** concentrar-se na reunião (desligar celular, esquecer o que ocorre lá fora);
- h)** perguntar e esclarecer qualquer dúvida;

- i) manter a calma e o bom humor;
- j) não interromper;
- k) ser impessoal;
- l) nas reuniões com participantes de áreas diversas, evitar tratar assuntos específicos de uma determinada área.

2.2.2 Pontos negativos de uma reunião

- a) atrasos;
- b) saídas antecipadas;
- c) interrupções por terceiros/telefone;
- d) discussões desnecessárias;
- e) resistência a mudanças;
- f) participantes despreparados;
- g) prolongamento – desrespeito à hora de início e término previstos;
- h) falas simultâneas.

2.2.3 Estereótipos dos participantes

Há alguns estereótipos que quando identificados ajudam na condução da reunião:



Fique atento a esses estereótipos; normalmente em toda reunião é possível identificá-los. Em alguns casos, nós mesmos representamos alguns desses papéis... É extremamente importante que o responsável por conduzir a reunião consiga bloquear ou minimizar a ação desses participantes. Caso contrário, muito será perdido durante a reunião.

- a) o comentarista;
- b) o desenhista;
- c) o engraçadinho;
- d) o sabido;
- e) o chato;
- f) o batedor de papo lateral;

- g) o que concorda com tudo;
- h) o puxa-saco;
- i) o dorminhoco;
- j) o rei da batucada.

2.3 Joint Application Design (JAD)

Tem como objetivo principal acelerar e consolidar o desenvolvimento de aplicações. Foi desenvolvida no final da década de 1970, início da década de 1980, pela IBM® Canadá.

Características:

- a) reuniões substituem as entrevistas individuais;
- b) decisões baseadas em consenso;
- c) economia de 30% a 40% do tempo.

Benefícios:

- a) redução da necessidade de manutenção nos aplicativos desenvolvidos;
- b) redução de custos;
- c) maior satisfação dos usuários;
- d) maior entrosamento entre a área de Sistemas de Informações e os departamentos usuários;
- e) menor necessidade de modificações durante o processo de desenvolvimento;
- f) nivelamento das expectativas de ambas as partes.

Componentes:

- a) **patrocinador executivo:** é o executivo que banca o projeto. Detém autoridade formal sobre as áreas de negócio afetadas. Estabelece os objetivos e diretrizes. Ele abre a sessão inicial demonstrando o apoio e a importância dados pela administração superior ao projeto;

- b) gerente de projeto:** normalmente é o líder do time de desenvolvimento da aplicação. Ele gerencia o dia a dia do projeto respondendo às questões sobre o escopo, prazo, coordenação dos recursos. Nas sessões atua como membro da equipe;
- c) líder de sessão – facilitador:** guia imparcial do grupo. Ele não contribui com informações; apenas conduz os participantes ao longo da agenda, garantindo que todos sejam ouvidos e que haja consenso. Responsável por identificar as questões que devem ser discutidas na reunião e aquelas que devem ser encaminhadas para investigação e discussão futuras;
- d) documentador:** registra e publica os procedimentos da reunião. Ele não contribui com informações;
- e) participantes:** são os clientes da área de negócio que são afetados de forma direta ou indireta pelo projeto. Eles são especialistas em suas áreas e podem tomar decisões sobre sua área de trabalho. Eles são a fonte de informações sobre as regras de negócio em que atuam;
- f) observadores:** geralmente são membros do time de desenvolvedores envolvidos no projeto. Eles ficam sentados atrás dos participantes e observam, sem participação, os procedimentos da reunião.

2.3.1 JAD – Organização

O processo é bem estruturado - cada sessão tem uma finalidade previamente definida. O líder conduz o grupo para o objetivo, seguindo os passos da agenda. *Flip-charts* são colocados nas paredes de forma que todos vejam os resultados.

Fases do JAD:

- a) Abertura – agenda:** apresentação dos participantes. Estabelecimento de conduta dos participantes. Apresentação da agenda. Palavras do executivo patrocinador (visão estratégica/objetivos do projeto).
- b) Desenvolvimento:** apresentação do tópico da pauta. Discussão dos assuntos pertinentes. Utilização de *brainstorming*, consenso ou votação. Obtenção e registro dos acordos e mudança para o próximo tópico.
- c) Fechamento:** resumo dos tópicos apresentados. Resumo das soluções aprovadas. Distribuição de tarefas e anotação de tendências. Avaliação da sessão. Agendamento da próxima data.

Tipos de sessão:

- a) **Planejamento:** é uma sessão inicial do projeto que visa abordar objetivos, metas, escopo e estratégias. Máximo de vinte pessoas.
- b) **Processos, eventos, dados:** dependem das técnicas de análise (serão estudadas em breve). Visam definir os componentes de *software* do sistema. Gerentes e supervisores devem estar presentes. No máximo de dez pessoas.
- c) **Interface:** definição de telas, relatórios e interfaces. Apresentação de protótipos. Supervisores e pessoal operacional. Máximo de cinco pessoas.

Veja no Quadro 2.1 uma comparação do JAD com técnicas tradicionais de levantamento de dados.

Quadro 2.1: Comparação de técnicas de levantamento de requisitos	
Levantamento Tradicional	Utilizando JAD
Diversas reuniões e individuais. Maior tempo para <i>feedback</i> .	Poucas reuniões e mais produtivas. Acelera comunicação e reduz tempo de resposta.
Usuários e desenvolvedores isolados.	Comprometimento de todos com o sucesso do sistema. Maior envolvimento dos usuários.
Problemas clássicos “eu mudei de ideia” diversas reuniões para obter consenso na mudança.	Estrutura do JAD facilita a interação entre os participantes.

Fonte: Elaborada pelo autor Gustavo Menezes

2.4 Análise essencial

Antes de começarmos definitivamente com técnicas orientadas a objetos e ao uso da *Unified Modeling Language* (UML), vamos fazer um pequeno estudo com relação à Análise Essencial de Sistemas. Embora seja uma técnica pouco utilizada atualmente, grande parte dos sistemas existentes e sua documentação está associada à Análise Essencial ou Estruturada.

A Análise Essencial pode ser vista como uma evolução da Análise Estruturada por adicionar a preocupação com o controle. Usa uma lista de eventos como base para o particionamento do sistema. Os eventos geram fluxos de dados (estímulos) para o sistema; o modelo essencial é construído sem considerar restrições de implementação (assume uma tecnologia perfeita) – o que permite uma solução ideal para o problema.

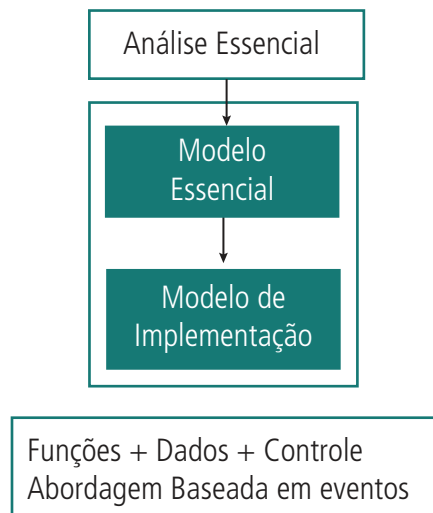


Figura 2.1: Análise Essencial de Sistemas

Fonte: Adaptada de Pompilho (2002, p. 192)

2.5 Estrutura da análise essencial

Para facilitar o processo de modelagem de sistemas a partir da Análise Essencial, a estrutura desse processo de desenvolvimento foi dividida nos modelos ambiental (relação do sistema com o ambiente externo – usuários e outros sistemas) e comportamental (relacionado ao comportamento interno do sistema). A seguir são apresentados os principais documentos ou diagramas que são utilizados em cada um dos modelos:

a) Modelo ambiental

Define a fronteira entre o sistema e o ambiente:

- a) descrição do propósito do sistema;
- b) diagrama de contexto;
- c) lista de eventos;
- d) modelo de Entidade e Relacionamento (conceitual);
- e) dicionário de dados.

b) Modelo comportamental

Descreve o comportamento interno do sistema em função de estímulos que ocorrem no ambiente externo:

- a) DFD – Diagrama de Fluxo de Dados;
- b) DER – Diagrama de Entidade e Relacionamento;
- c) DTE – Diagrama de Transição de Estado;
- d) dicionário de dados;
- e) especificação de processos, etc.

No primeiro momento, considera-se uma tecnologia ideal e sem limitações, em que:

- a) os custos, o consumo e o desgaste são zero;
- b) a capacidade de armazenamento de dados do sistema é infinita;
- c) a velocidade dos processadores é infinita;
- d) zero erro (não ocorrem falhas);
- e) é constituída basicamente por duas fases (modelo ambiental e comportamental).

2.6 Modelo ambiental

Tem-se a especificação macro do sistema (como se fosse uma caixa-preta).

2.6.1 Declaração de objetivos

- a) deve estar resumida a um parágrafo;
- b) ser global, especificando o principal propósito da criação do *software*;
- c) não cabe neste ponto estabelecer uma lista textual com todas as funcionalidades esperadas no sistema.

2.6.2 Diagrama de contexto

- a) reflete graficamente a relação do sistema com o meio ambiente onde está inserido;
- b) essa relação dá-se por meio de recebimento de estímulos do meio ambiente, os quais ativam processos, e esses, por sua vez geram respostas externas ao sistema, ou seja, resposta ao meio ambiente;

2.6.3 Lista de eventos

- a) especificação das atividades (processos) essenciais que o sistema terá;
- b) essas atividades são ativadas por estímulos, executam processamento e geram respostas;
- c) não há uma precedência estabelecida para a elaboração da lista de eventos e o diagrama de contexto.

São atividades que podem estar acontecendo em paralelo, porém devem estar consistentes.

2.7 Modelo comportamental

O analista irá detalhar como cada atividade existente na lista de eventos se comporta (como ela deve funcionar). Será feito um modelo de dados sobre o qual o sistema atuará. Cria-se um dicionário de dados e um DFD. Possui os seguintes diagramas:

a) Diagrama de fluxo de dados particionado por evento

Para cada item da lista de eventos, o analista de sistemas fará um Diagrama de Fluxo de Dados, representando de forma gráfica, individualmente, cada evento existente no sistema.

b) Diagrama Entidade Relacionamento

Para a modelagem de dados o analista fará inicialmente o DER (modelagem lógica) e logo após irá gerar DER (modelagem física).

c) Diagrama hierárquico de macroatividades

- propicia uma visão sintética única do sistema;
- aglutina-se às funcionalidades existentes na lista de eventos de acordo com os assuntos que trata;
- aglutina-se por afinidade aos DFDs particionados por evento;
- finalidade documental e possibilidade de examinar e definir interfaces.

d) Dicionário de dados

- os dados referenciados na construção do sistema devem ter sua definição no dicionário de dados;
- padrões para construção do dicionário de dados.

2.8 Lista de eventos

Evento é uma frase que expressa um acontecimento externo ao sistema, que gera um estímulo no sistema, que produz uma resposta. O evento é a causa do surgimento do estímulo. Portanto, um evento pode gerar vários estímulos partindo de diferentes entidades externas. Os eventos são classificados pelo estímulo que eles provocam no sistema. Vejamos quais são eles:

- a) evento orientado por fluxo de dados (F)** – aquele em que o estímulo é a chegada do fluxo de dados enviados por uma entidade externa.

O evento é escrito da seguinte forma:

Sujeito + Verbo transitivo na voz ativa + Complementos verbais

Veja um exemplo na Figura 2.2 a seguir.

Evento	Fluxo de Dados
Cliente paga prestação	→ Dados do pagamento ← Recibo do cliente
Cliente cancela compra	→ Dados da Compra Resposta interna

Figura 2.2: Exemplo de evento

Fonte: Adaptado de Pompilho (2002, p. 158).

- b) evento orientado por controle (C)** – é aquele em que uma entidade externa envia um controle que muda o estado do sistema, ou um depósito de dados do sistema. Um evento orientado por fluxo de dados pode disparar internamente um evento de controle. Ex.: “Caixa emite N.F.” dispara “Nível de ressuprimento de estoque atingido”.

O evento deve seguir:

Sujeito + Verbo transitivo na voz ativa + Complementos verbais

Sujeito + Verbo na voz passiva

Veja um exemplo na Figura 2.3 a seguir.

Evento	Fluxo de Dados
Técnico Fecha OS	→ Dados da OS ← Resposta interna

Figura 2.3: Evento de controle

Fonte: Adaptado de Pompilho, 2002, p.158

- c) **evento temporal (T)** – ocorre quando é transcorrido um intervalo de tempo.

Modelo:

É hora de + Verbo no infinitivo + Complementos verbais

Veja um exemplo na Figura 2.4 a seguir.

Evento	Fluxo de Dados
É hora de enviar remessa bancária	← Dados da Remessa

Figura 2.4: Evento temporal

Fonte: Adaptado de Pompilho (2002, p. 158)

A Tabela 2.1 apresenta uma lista de eventos. Nessa tabela são listados os principais eventos do sistema, bem como o seu tipo (fluxo, controle ou temporal).

Tabela 2.1: Lista de eventos

Nº	Nome do Evento	Descrição do Evento	Estímulo	Tipo Estímulo	Ação ou Processo	Resposta
01	Cliente Reserva Quarto	Quando o cliente telefona, ou vem até o hotel, e pede para reservar um quarto, o funcionário executa um procedimento padrão.	Dados Cliente Reserva	F	Efetuar Reserva	Status Reserva

Fonte: Adaptada de Pompilho (2002. p. 158)

2.8.1 Como elaborar a lista de eventos

Nome do evento:

Nome + Verbo + Complemento

Ex.: Cliente reserva um quarto (Funcional)

É hora de cancelar a reserva (Temporal)

Descrição do evento:

Como acontece o evento?

Item facultativo (porém se não constar na lista de eventos, deverá constar no DFD particionado por eventos).

Estímulo:

É o fluxo de dados.

Ex.: Dados da Reserva/Cliente

Tipo Estímulo:

F – (fluxo de dados) quando a entidade externa envia dados para o sistema;

T – (temporal) quando as ações são do próprio sistema (o estímulo é interno ao sistema);

Nesse caso a coluna do estímulo deve ficar em branco ou com um traço (-).

E o nome do evento deve começar com “É hora de...” + complemento.

Resumo

Caro estudante, nesta aula conhecemos as principais técnicas de coleta de dados – aspecto fundamental no Projeto de Sistemas. As principais técnicas estudadas foram: Entrevistas, Reunião em Grupo (JAD) e *Brainstorming*. Além dessas técnicas, tivemos uma breve revisão da Análise Essencial de Sistemas.

Atividades de aprendizagem

Você foi convidado a informatizar uma empresa de assistência técnica de computadores. Durante a entrevista com o gerente foram obtidas as seguintes informações: um cliente, ao trazer um equipamento, é atendido pelo balconista. Se ele não for cadastrado, primeiramente deve-se realizar seu cadastro. Só então se verifica, pelo número de série, se o equipamento está em garantia. É disponibilizado pelo fabricante um sistema de consulta remota (via *socket*) que poderá ser utilizado com a implantação do novo sistema. Se for garantia, a mão de obra e peças serão pagas pelo fabricante; se não, ficará a cargo do próprio cliente. Em ambos os casos o balconista emite uma ordem de serviço com as solicitações do cliente. Uma via da OS fica com o cliente e a outra acompanha o equipamento que vai para oficina para diagnóstico. O técnico informa no sistema, registrando na OS, o(s) serviço(s) e peça(s) necessários no reparo, e a sua matrícula para identificar quem fez o orçamento. Em seguida emite o orçamento. O setor administrativo recebe em mãos do técnico os orçamentos pendentes e faz contato telefônico a fim de conseguir ou não a autorização para realização do reparo junto ao cliente ou fabricante, conforme o caso; em seguida, registra no sistema. Se houver aprovação, o setor administrativo devolve pessoalmente o orçamento para o técnico. Após executar o reparo, o técnico fecha a OS registrando no sistema a data de término do reparo e a sua matrícula (identifica quem fez o reparo). O equipamento fica disponível, aguardando o cliente. Quando o cliente retorna para buscar o equipamento, ele é novamente atendido pelo balconista que, através do número da OS, localiza manualmente o equipamento. O cliente faz o pagamento, se houver, ficando registrados no sistema a data da entrega e o valor pago.

1. Responda às perguntas:

- a) Como fica a lista de eventos?
- b) E o DER (Diagrama de Entidade Relacionamento)?
- c) Quais são as principais funcionalidades?

- d) Quem são os usuários e em que momentos eles irão interagir com o sistema?
 - e) Quais serão as entradas e saídas de dados?
2. Descreva os princípios básicos de quatro técnicas de coleta de dados.
 3. Com relação à técnica de entrevista, descreva como cada uma das quatro etapas deve ser realizada (preparação prévia, estrutura, forma de condução e documentação).
 4. Enumere pelo menos cinco estereótipos de participantes e descreva o perfil de cada um deles.
 5. O que é JAD? Quais são as suas características?
 6. Descreva os componentes do JAD e a função de cada um deles.
 7. Monte um quadro comparativo entre o levantamento de requisitos tradicional e a técnica JAD.
 8. Observe os conceitos de levantamento de requisitos, analise as sentenças e marque V ou F.

[]	Podem ser consideradas como técnicas de levantamento de requisitos as entrevistas, a pesquisa em manuais técnicos, aplicação de questionário, o <i>Walkthrough</i> e as reuniões em grupo.
[]	Para se obter um bom produto final de <i>software</i> é necessário iniciar rapidamente a codificação do sistema evitando dessa forma que o usuário possa inserir novas demandas de funcionalidades.
[]	O <i>Joint Application Design</i> (JAD) tem como objetivo principal acelerar e consolidar o desenvolvimento de aplicações. Ele favorece as decisões em consenso.
[]	Os participantes de uma reunião JAD têm os papéis bem definidos. São eles: patrocinador executivo, gerente de projeto, líder de sessão, documentador, participantes e observadores.

Você foi convidado a informatizar uma empresa que deseja realizar venda de CDs pela internet. Durante a entrevista com o idealizador do *site*, foram obtidas as seguintes informações. O *site* irá disponibilizar diversos títulos de CDs de diversas gravadoras. O *site* terá vários tipos de usuários. O primeiro é o administrador, que será o responsável em cadastrar todos os autores, gravadoras e títulos de CD's, além de informar preços e promoções. O segundo, o internauta, que irá acessar o *site*. Qualquer internauta poderá consultar o *site*. Mas para efetuar uma compra, o internauta deverá primeiro realizar o seu cadastramento. A venda po-

derá ser feita por cartão, autorizado via Sitef (Sistema de Transferência Eletrônica de Fundos), o qual irá gerar os pedidos solicitados quitados, ou através emissão de boleto gerando pedidos solicitados pendentes. Um terceiro usuário (financeiro) será o responsável por processar a quitação dos pedidos solicitados pendentes a partir da remessa de baixa de títulos enviadas pelo banco cobrador dos boletos. Um novo usuário responsável pela expedição das mercadorias busca informação dos pedidos pendentes quitados via cartão e boleto diariamente, a fim de emitir as notas fiscais e processar a baixa no estoque.

9. Responda às perguntas:

- a)** Como fica a lista de eventos
- b)** E o DER (Diagrama de Entidade Relacionamento)?
- c)** Quais são as principais funcionalidades?
- d)** Quem são os usuários e em que momentos eles irão interagir com o sistema?
- e)** Quais serão as entradas e saídas de dados ?

Registre suas respostas num arquivo de texto e poste-o no AVEA.

Aula 3 – Análise orientada a objetos – introdução à UML

Objetivos

Conhecer os principais conceitos referentes à orientação a objetos.

Utilizar alguns dos principais diagramas da UML para modelagem de sistemas.

3.1 Características

É uma nova uma forma de abordar (entender e construir uma solução) de um problema, baseada no paradigma da orientação a objetos. É uma evolução da Análise Essencial e de outras técnicas de análise.

Suas principais características são:

- a) **melhoria na interação entre o analista e o usuário:** utiliza métodos naturais de organização do pensamento humano, como a analogia;
- b) **aumento da consistência interna da análise:** dados e processos são projetados conjuntamente;
- c) **representação explícita dos pontos comuns:** uso de herança para identificar atributos e métodos comuns com especificações que suportam alterações de alto nível conceitual (propagação);
- d) **reutilização.**

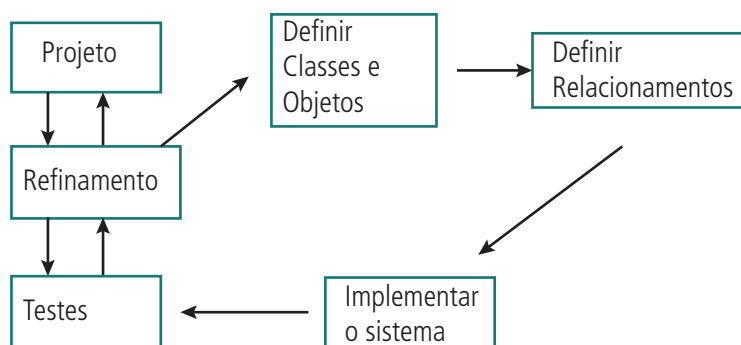


Figura 3.1: Etapas do processo de desenvolvimento de software OO

Fonte: Elaborada pelo autor Gustavo Menezes

3.2 Fundamentos da orientação a objetos

Orientação a objetos é uma técnica de desenvolvimento de sistemas muito utilizada atualmente. O seu principal objetivo é procurar facilitar o processo de modelagem de um sistema. Uma das grandes dificuldades nesse processo é fazer com que o analista de sistemas consiga capturar o que o sistema deverá fazer através das técnicas de levantamento de dados vistas anteriormente. Antes de o analista partir para a programação/implementação do sistema, é necessário que ele consiga construir um modelo do sistema para, em seguida, partir para a programação. A análise orientada a objetos permite não apenas construir esse modelo, como também, através de Linguagens de Programação Orientadas a Objetos, desenvolver o sistema. A seguir destacam-se algumas características da orientação a objetos:

- a) qualquer coisa é um objeto;
- b) objetos realizam tarefas através da requisição de serviços a outros objetos;
- c) cada objeto pertence a uma determinada classe. Uma classe agrupa objetos similares;
- d) a classe é um repositório para comportamento associado ao objeto;
- e) classes são organizadas em hierarquias.

Um sistema é visto como um conjunto de entidades interconectadas chamadas objetos. Cada objeto é responsável por realizar uma tarefa específica. É através da interação entre objetos que todas as funcionalidades do sistema são realizadas.

3.3 Conceitos e princípios da OO

Para começar a entender a orientação a objetos (OO), alguns conceitos são fundamentais:

- a) **classe:** é um molde para objetos. Diz-se que um objeto é uma instância de uma classe. É uma abstração das características importantes de um grupo de coisas do mundo real. Classe possui membros (atributos e métodos).
- b) **objeto:** é a instância de uma classe. Composto por:
 - **atributos:** memória onde valores podem ser armazenados e modificados ao longo da vida do objeto;

- **comportamento:** conjunto de ações predefinidas (métodos) através das quais o objeto responderá às mensagens e desempenhará papéis (funções).

OBJETO = DADOS + PROCESSOS

Um objeto é criado por alguém, tem uma vida e morre ou é destruído por alguém.

Objeto como abstrações:

Dependendo do contexto, um mesmo conceito do mundo real pode ser representado por diferentes abstrações.

- carro (para uma transportadora de cargas);
- carro (para uma fábrica de automóveis);
- carro (para um colecionador);
- carro (para uma empresa de kart);
- carro (para um mecânico).

Objetos são abstrações de entidades que existem no mundo real.

Classes são definições estáticas, que possibilitam o entendimento de um

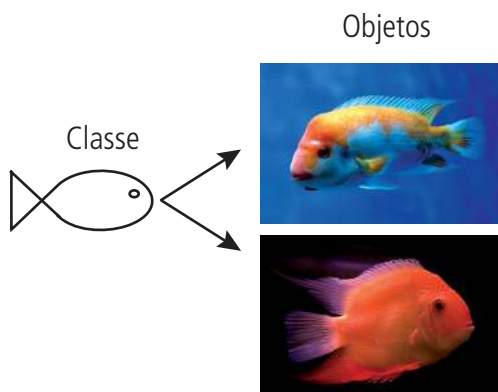


Figura 3.2: Relação entre classe e objeto

Fonte: Elaborada pelo autor Edson Marchetti da Silva

Para que um objeto realize alguma tarefa, ele deve receber um estímulo. Um objeto é uma entidade que representa uma abstração de algum requisito do mundo real e pode responder a estímulos a ele enviados. Quando ocorre um estímulo em um objeto, pode-se dizer que o objeto está recebendo uma mensagem.

Encapsulamento é uma forma de restringir o acesso ao comportamento de um objeto. Graças ao encapsulamento, o projetista de uma classe pode “esconder” a implementação de determinados métodos ou restringir o acesso a determinados atributos.

Na **herança**, classes semelhantes são agrupadas em hierarquias. Cada classe em um nível da hierarquia herda as características das classes nos níveis acima. A principal vantagem desta característica é o reaproveitamento de código.

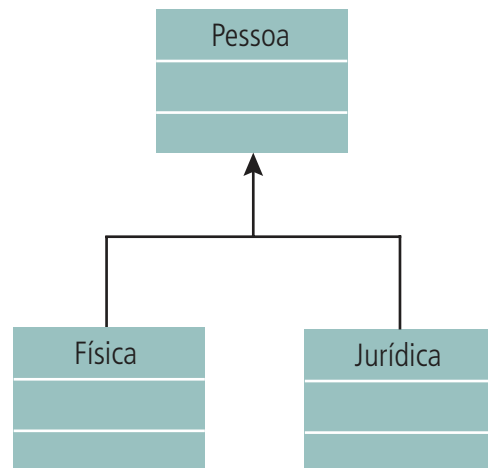


Figura 3.3: Exemplo 1 - Herança

Fonte: Elaborada pelo autor Gustavo Menezes

Hierarquia de generalização-especialização: cada classe herda as variáveis e métodos definidos em classes superiores (superclasse). Cada classe pode e deve adicionar variáveis e estender métodos herdados. A subclasse só justifica sua existência a partir do momento em que acrescenta algo (nem que seja um atributo e um processo) em relação às propriedades herdadas. As subclasses são sempre versões mais especializadas de suas superclasses. Sem a herança, cada classe seria uma unidade independente e desenvolvida a partir do zero. A execução dos métodos se dá pela procura “*bottom-up*”. Não encontrou no filho, procura-se no pai... (*Override*).

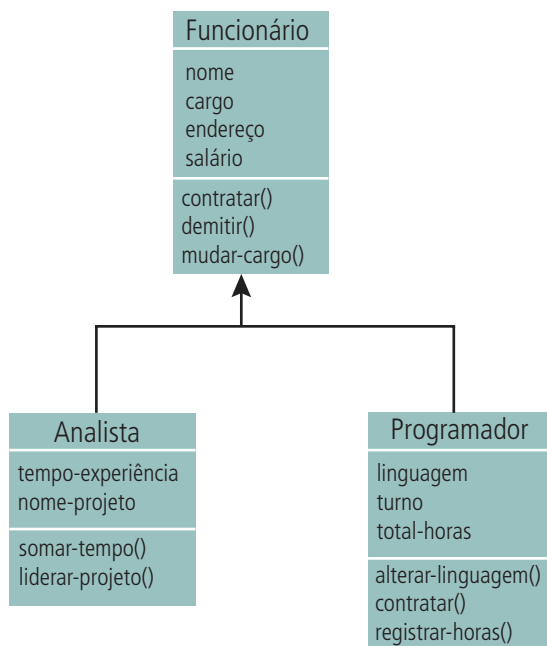


Figura 3.4: Exemplo 2 - Herança

Fonte: Elaborada pelo autor Edson Marchetti da Silva

Composição: Identificar classes que necessitam de objetos de outras classes para ser implementadas. Relacionamento **todo** (que usa) / **parte** (o que é usado) ou “**possui-um**”.

Exemplos:

Pessoa **possui uma** data de nascimento.

Janela **possui um** título e dois botões.

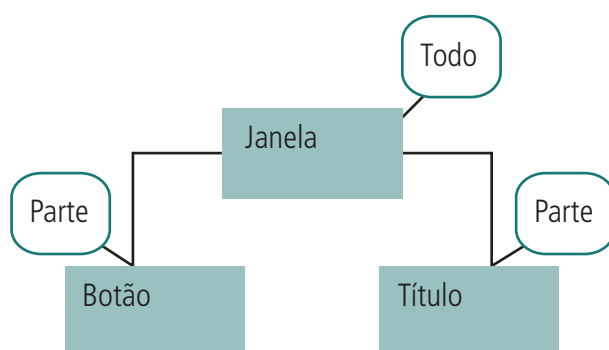


Figura 3.5: Composição

Fonte: Elaborada pelo autor Gustavo Menezes

Agregação: O conceito de uma classe contém

Exemplo: Um controlador de temperatura **contém** um aquecedor.

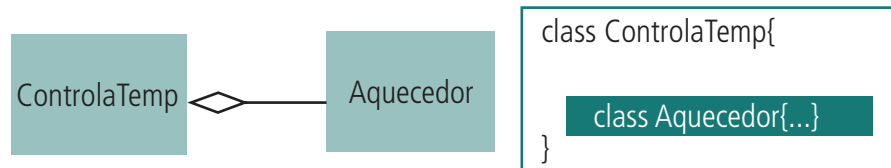


Figura 3.6: Exemplo de agregação

Fonte: Elaborada pelo autor Edson Marchetti da Silva

Composição X agregação: ambas se referem a relacionamentos de uso: um objeto é parte de outro objeto.

Agregação

relacionamento “contém-um”

implementação pode usar a definição de uma classe interna.

Composição

relacionamento “possui-um”;

implementação usa a definição de uma classe externa.

Resumo

Na nossa terceira aula começamos a estudar Análise Orientada a Objetos. Esse tema é sem dúvida nenhuma um dos mais importantes atualmente no Projeto de Sistemas. Aproveitamos também para iniciar o nosso estudo em Unified Modeling Language (UML), que pode ser traduzido como Linguagem de Modelagem Unificada. Nas próximas aulas iremos nos aprofundar nos principais diagramas disponíveis dessa linguagem.

Atividades de aprendizagem

1. Qual a principal diferença entre Classe e Objeto?
2. Agora que você já fez as disciplinas de Linguagem de Programação I e II, estabeleça quais as principais diferenças entre a Programação Estruturada e a Programação Orientada a Objetos.
3. Descreva vantagens e desvantagens em se usar:
 - Programação Estruturada
 - Orientação a Objetos.

Registre suas respostas num arquivo e poste-o no AVEA.

Aula 4 – *Unified Modeling Language* (UML)

Objetivos

Conhecer os principais diagramas da UML para o desenvolvimento de *software*.

4.1 O que é a UML?

O *Object Management Group* (OMG) é uma organização internacional que define padrões e conceitos para aplicações orientadas a objetos. Segundo a especificação da OMG:

- a) a UML é uma linguagem gráfica para visualizar, especificar, construir e documentar artefatos de um sistema *software*-intensivo.
- b) a UML oferece um modo padrão para descrever um sistema, incluindo tanto coisas conceituais, tais como modelos de processo e funções de sistema, como coisas concretas, tais como declarações de linguagens de programação, esquemas de bases de dados e componentes reusáveis de *software*.

4.1.1 Finalidade

- a) a UML possui um conjunto de diagramas que permite concentrar esforços em diferentes aspectos do sistema ao mesmo tempo.
- b) a UML torna mais fácil a comunicação entre todos os envolvidos no desenvolvimento de sistemas.

4.1.2 Classe X objeto

Classes:

- a) são tipo abstrato de dados;
- b) são como plantas de objetos;
- c) contêm métodos e dados.

Objetos:



UML é uma linguagem de modelagem unificada que rapidamente se tornou um padrão para a construção de *software* orientado a objetos. Padronizada pela *Object Management Group* (www.omg.org).

- a) caso particular de uma classe;
- b) instâncias de uma classe;
- c) contêm métodos e dados;

4.2 Estrutura da UML

A UML 2.0 define 13 diagramas básicos, distribuídos em dois conjuntos gerais:

a) diagramas de Modelagem Estrutural

- definem a arquitetura estática de um modelo;
- descrevem “coisas” que fazem parte de um modelo (classes, objetos, interfaces e componentes físicos);
- modelam a relação e interdependência entre elementos.

b) diagramas de Modelagem Comportamental

- capturam as variedades de interação e estados instantâneos em um modelo, à medida que este “executa” no tempo.

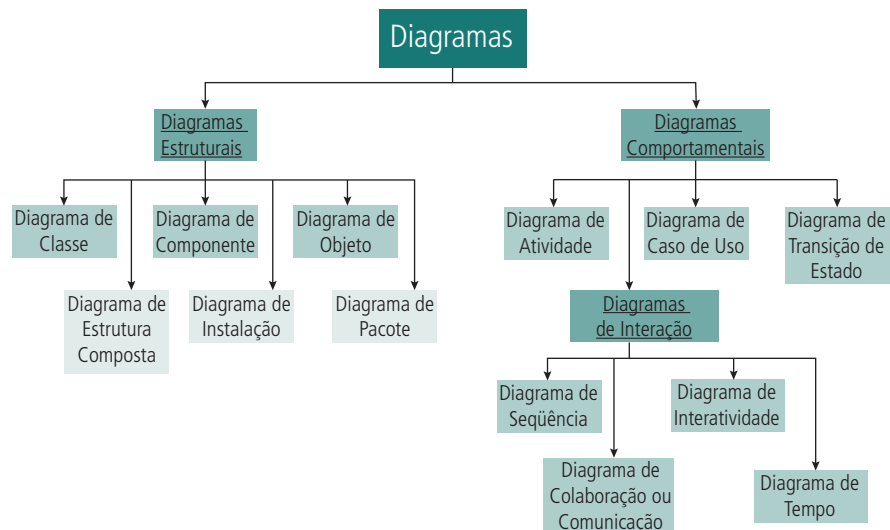


Figura 4.1: Principais diagramas da UML

Fonte: <http://www.microsoft.com/br/net/>

Nas próximas seções é feita uma breve descrição dos diagramas existentes na UML. Na aula seguinte, os principais diagramas serão discutidos com mais detalhes.

4.3 Diagrama de atividade

O propósito principal da elaboração de um diagrama de atividade é mostrar, em forma de modelos, os fluxos principais, alternativos, de exceção e concorrentes ao fluxo principal. Esses fluxos são apresentados nas descrições dos casos de uso. Portanto, podemos definir atividades e diagrama de atividade da seguinte maneira: o diagrama de atividade descreve a sequência de atividades guiadas por um cenário estabelecido.

Uma atividade é estar fazendo algo, por exemplo, a efetivação de uma reserva, consulta de voos, entre outros.

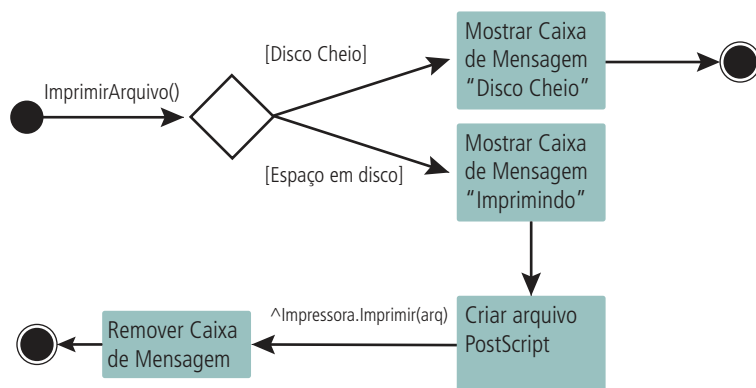


Figura 4.2: Modelo de um Diagrama de Atividade

Fonte: <http://www.microsoft.com/br/net/>

4.4 Diagrama de caso de uso

Os diagramas de caso de uso são a forma visual de apresentar o conjunto de atores, funcionalidades e relações entre eles, diante de um determinado objetivo, e provenientes dos passos de suas identificações. Na prática esse diagrama é elaborado para ser um instrumento de breve leitura, já que estamos transportando para um desenho tudo que identificamos e descrevemos sobre os requisitos do cliente, podendo, assim, representar melhor a complexidade do sistema.

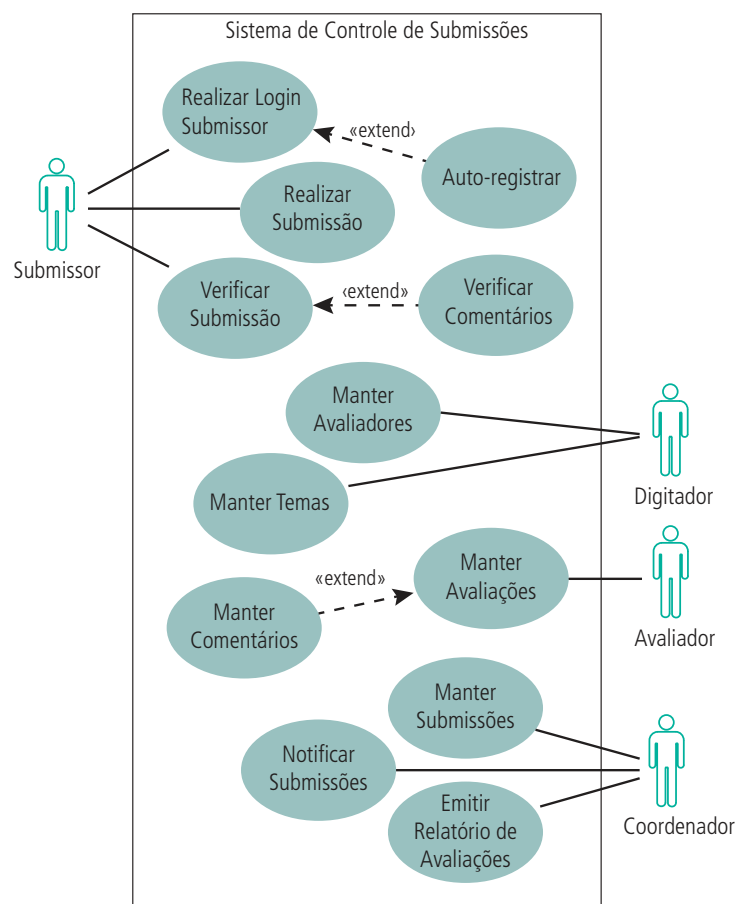


Figura 4.3: Caso de uso

Fonte: Elaborada pelo autor Edson Marchetti da Silva

4.5 Diagrama de transição de estado

O propósito de um diagrama de transição de estado é mostrar em forma de desenho os possíveis estados de um objeto em particular. Através dele podemos visualizar a sequência de estados que um objeto ou uma interação percorre durante sua vida em resposta a estímulos recebidos, junto com suas próprias ações e respostas. Portanto, podemos definir que o diagrama de transição de estado representa os eventos possíveis para somente um objeto individual, e as ações tomadas por esse objeto em resposta a esses eventos.

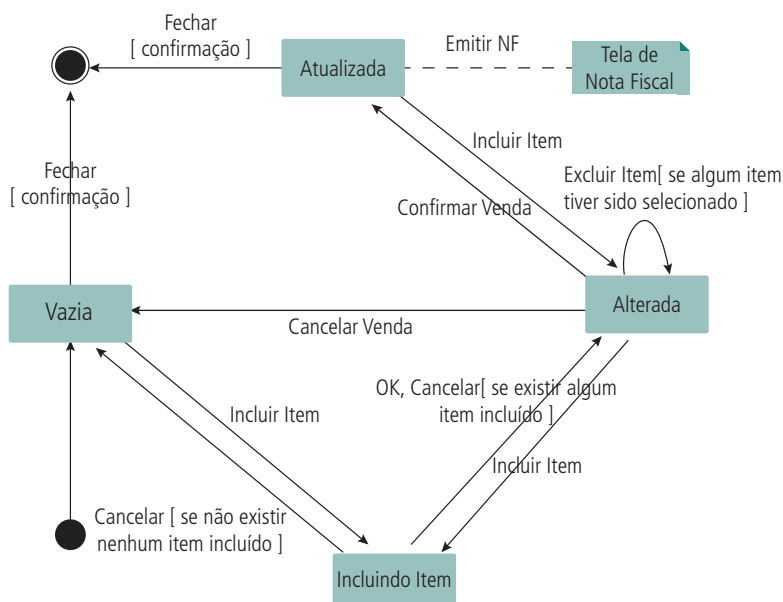


Figura 4.4: Diagrama de transição de estado

Fonte: <http://www.microsoft.com/br/net/>

4.6 Diagrama de sequência

O diagrama de sequência é o que representa as comunicações efetuadas pelas classes do sistema, conforme um cenário específico, em um determinado momento. Em outras palavras, esse diagrama tem o objetivo de exibir visualmente a sequência das ações que o *software* executará diante dos comandos dados por um ator. O diagrama de sequência pode ser elaborado com base nos diagramas de caso de uso e de classes, definidos nas fases anteriores do projeto. Porém, para que um diagrama de sequência seja elaborado, ao menos o diagrama de classes deve ter sido modelado anteriormente.

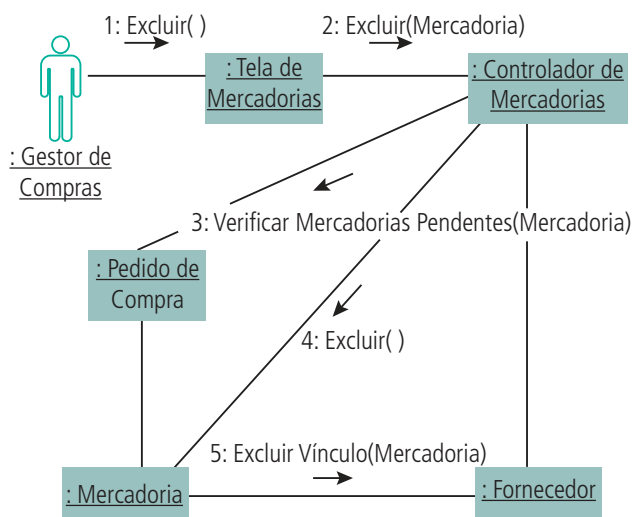


Figura 4.5: Diagrama de sequência

Fonte: <http://www.microsoft.com/br/net/>

4.7 Diagrama de colaboração ou comunicação

O diagrama de colaboração tem o mesmo objetivo do diagrama de sequência, ou seja, ele representa os aspectos comportamentais do sistema, trazendo informações sobre como as interações são efetuadas entre os objetos. As diferenças entre o diagrama de sequência e o de colaboração é que este não representa o tempo como uma dimensão separada. As execuções são exibidas em conjunto, independentemente do tempo de execução de cada objeto.

4.8 Diagrama de classe

O diagrama de classe é um dos mais importantes num processo de desenvolvimento de *software* que modela classes, relacionamentos e cardinalidades, os quais, em conjunto, apresentam a organização de estrutura de um sistema.

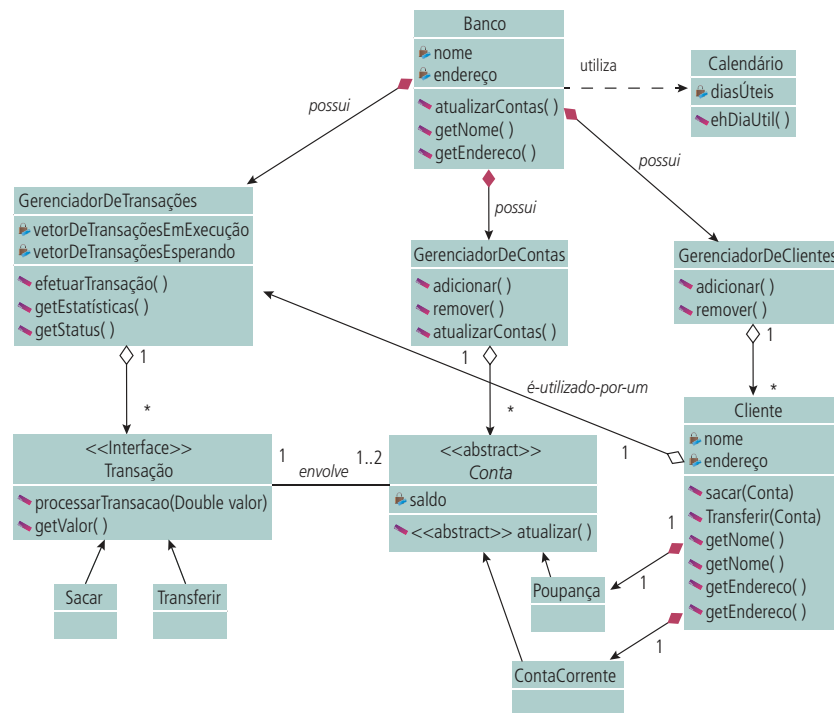


Figura 4.6: Diagrama de classe

Fonte: <http://www.microsoft.com/br/net/>

4.9 Diagrama de componente

O propósito da elaboração de um diagrama de componente é mostrar em forma de desenho a arquitetura de um sistema de *software* através de seus componentes, explicitando principalmente as suas dependências. O diagrama de componente ajuda-nos a entender como os diversos componentes

do sistema interagem entre si. Portanto, podemos definir que o diagrama de componente apresenta a arquitetura física e as dependências entre vários componentes do *software*.

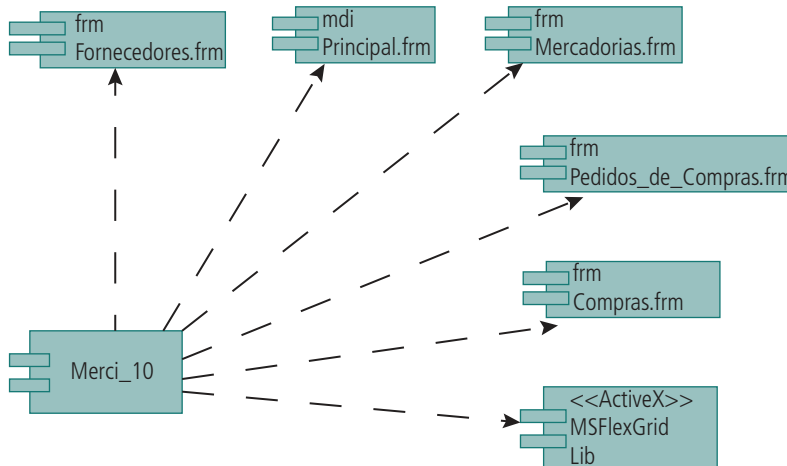


Figura 4.7: Diagrama de componente

Fonte: <http://www.microsoft.com/br/net/>

4.10 Diagrama de objeto

Desde o início do desenvolvimento de um sistema orientado a objetos, pensamos não só em definições e regras de negócios, mas também em técnicas de modelagem que auxiliem no entendimento e definição das técnicas de implementação a serem realizadas e utilizadas. O diagrama de objeto é definido como um tipo de modelo em que são representados os objetos que compõem o diagrama e interagem entre si num sistema orientado a objetos.

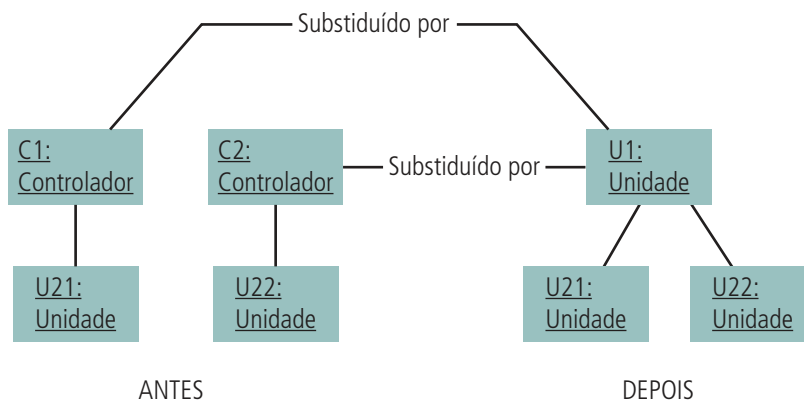


Figura 4.8: Diagrama de objeto

Fonte: <http://www.microsoft.com/br/net/>

4.11 Diagrama de instalação

O propósito da elaboração de um diagrama de instalação é mostrar como deve ser feita a organização do *hardware* e a comunicação com o *software* nos diversos dispositivos. Esse diagrama apresenta vários dispositivos de *hardware* e interfaces físicas tais como: processador, impressora, memória, e disco.

dd Network Model

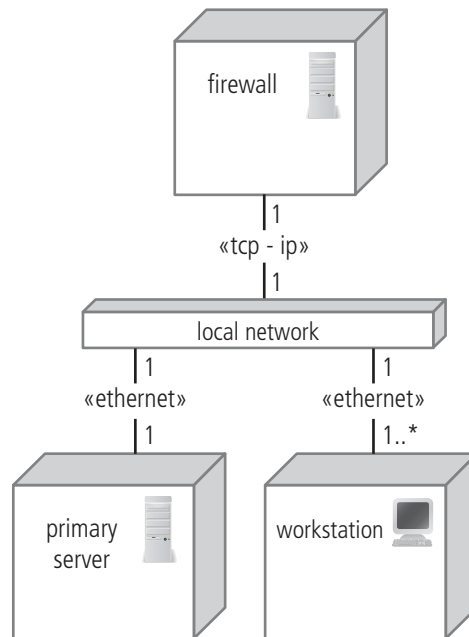


Figura 4.9: Diagrama de instalação

Fonte: <http://www.microsoft.com/br/net/>

4.12 Diagrama de pacote

O propósito da elaboração de um diagrama de pacote é mostrar como os elementos podem ser organizados dentro de um sistema. Nota: os elementos poderão ser considerados como classes e/ou pacotes. A vantagem de termos a representação de elementos em pacotes é que quebramos um sistema complexo em partes menores, fornecendo uma visão de quantas e como se relacionam as várias partes do sistema.

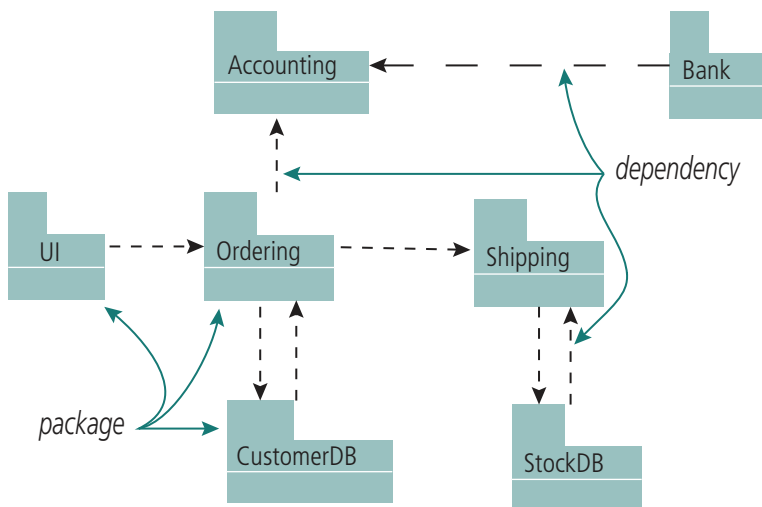


Figura 4.10: Diagrama de pacote

Fonte: <http://www.microsoft.com/br/net/>

Resumo

Prezado estudante, nesta aula conhecemos conceitos referentes à orientação a objetos e utilizamos alguns dos principais diagramas da UML para modelagem de sistemas.

Atividades de aprendizagem

1. O diagrama UML mais indicado para representar passo a passo o fluxo de eventos principais de um caso de uso de um *software* orientado a objetos é o diagrama de:
 - a) Caso de uso.
 - b) Eventos e transição.
 - c) Atividades.
 - d) Classe.
 - e) Fluxo de dados.
2. Explique o que é e para que serve a UML. Dê exemplos de pelo menos três diagramas estruturais e três diagramas comportamentais.

Registre suas respostas num arquivo e poste-o no AVEA.

Aula 5 – Diagrama de caso de uso

Objetivos

Analisar e criar diagramas de caso de uso para o sistema.

Analisar e criar diagramas de classe.

5.1 O que é diagrama de caso de uso?

É um diagrama que descreve uma interação entre um usuário e um sistema. Modela o sistema através de dois tipos de cenário: normal e alternativo.

Ex.: uma compra na *web* com cartão de crédito.

Cenário normal:

O internauta navega no catálogo de itens e adiciona os itens desejados à sua cesta de compras. Ele fornece dados de endereço de entrega, dados do cartão de crédito e confirma a compra. O sistema verifica autorização do cartão e confirma a venda, envia *e-mail* de confirmação.

Cenário alternativo:

E se o cartão de crédito não for aceito?

O diagrama de caso de uso pode ser entendido também como um conjunto de cenários “amarrados” por um objetivo comum de um usuário.

5.2 Maneira de documentar um caso de uso

O caso de uso pode ser representado por meio de uma ficha com os seguintes campos:

- a) nome de referência:** UC001 – Nome do caso de uso (verbo no infinitivo);
- b) descritivo:** uma breve descrição do que trata o caso de uso;
- c) condições:** informa o que é necessário acontecer para que o caso de uso se inicie;

- d) atores:** informar quais são os envolvidos;
- e) cenário principal:** descrição que apresente o cenário perfeito, sem exceções. Verbos no presente do indicativo ou substantivos. Ex.: registra, compra, seleciona etc;
- f) cenário alternativo:** situação que represente uma exceção do cenário principal;
- g) requisitos especiais:** qualquer situação não contemplada anteriormente (Requisitos implícitos: tempo de resposta, precisão etc.);
- h) dados:** tipos de dados que foram encontrados durante a descrição do caso de uso (características, tamanho etc.);
- i) observação:** _____;
- j) analista de negócio:** _____;
- k) entrevistado:** _____;
- l) área:** _____;
- m) data:** __/__/__;
- n) versão:** _____.

5.3 Exemplo de um caso de uso

Como exemplo de um caso de uso vamos considerar o processo de compra de um produto na *web* com cartão de crédito:

a) condição (opcional)

Internauta deverá ser autorizado com nome e senha.

b) cenário principal

1. o cliente navega pelo catálogo e seleciona itens a serem comprados;
2. o cliente confirma as compras;
3. o sistema apresenta o total da compra;
4. o cliente informa endereço de entrega e opções de pagamento;
5. o cliente informa dados do cartão;

- 6. o sistema autoriza a compra;
- 7. o sistema manda confirmação por *e-mail*.

c) cenário alternativo

Alternativa 1 – Cliente regular

- (a) o sistema mostra o local de entrega e os quatro últimos dígitos do cartão.
- (b) o cliente pode aceitar ou reescrever dados. Vai para o cenário principal do item 6.

Alternativa 2 – Falha na autorização

Sistema permite ao cliente resubmeter a informação do cartão e tentar novamente.

5.4 Representação gráfica de um caso de uso

Além das fichas descritas anteriormente, um diagrama de caso de uso também possui uma representação gráfica. A seguir são apresentadas as principais características dessa representação.

Componentes:

- a) ator** – representa o papel que o usuário está desempenhado ao interagir com o sistema. Simbolizado por um boneco;
- b) caso de uso** – representa um cenário;
- c) reta** - conecta o ator ao caso de uso;
- d) linha tracejada** - entre casos de uso (*include extends*);
- e) seta**, com a ponta não hachurada, entre casos de uso (generalização).

Associação:

- a) de inclusão (*include*)** - ocorre quando existe uma parte do comportamento que é semelhante em mais de um caso de uso.

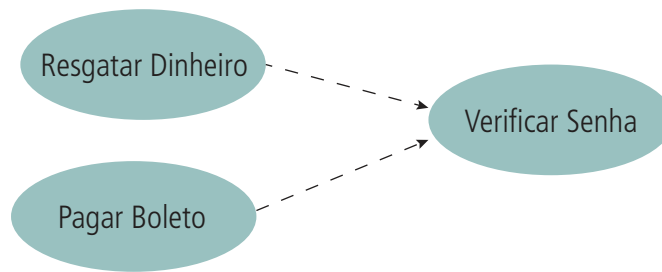


Figura 5.1: Exemplo de associação

Fonte: Elaborada pelo autor Edson Marchetti da Silva

- b) de generalização** - usada quando existem casos de uso semelhantes, só que um faz um pouco mais do que o outro. É comum utilizar generalização quando o objetivo é descrever uma variação do comportamento normal.

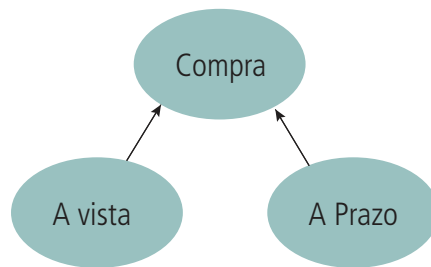


Figura 5.2: Exemplo de generalização

Fonte: Elaborada pelo autor Edson Marchetti da Silva

- c) de extensão (*extends*)** - semelhante à de generalização, mas possui mais regras acrescentando comportamentos adicionais. Use *extend* quando descrever uma variação de um comportamento normal.

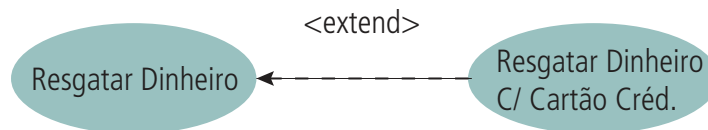


Figura 5.3: Exemplo de extensão

Fonte: Elaborada pelo autor Edson Marchetti da Silva

5.4.1 Diagrama de contexto

É o diagrama que mostra as interfaces do sistema com os diversos usuários e outros sistemas.

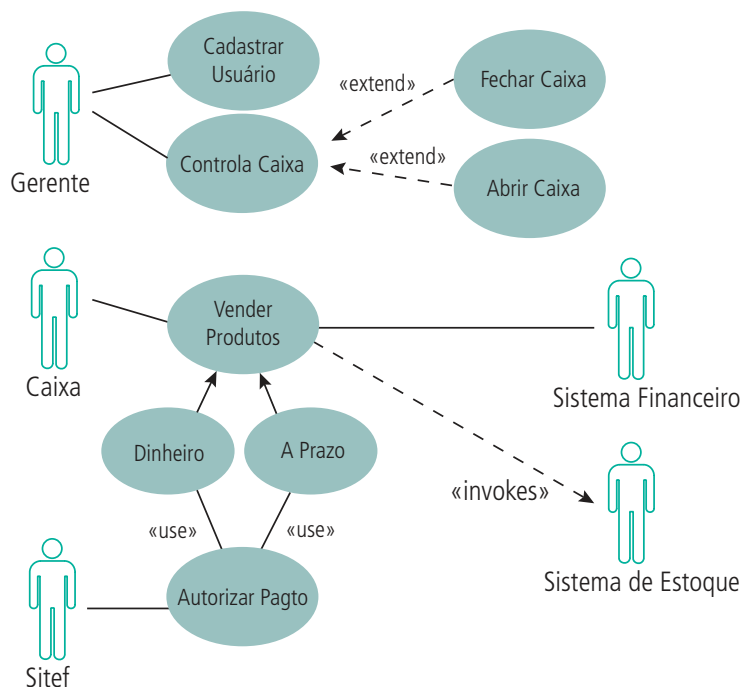


Figura 5.4: Diagrama de contexto

Fonte: Elaborada pelo autor Edson Marchetti da Silva (*software Enterprise architect 7.5*)

5.5 Conclusões sobre caso de uso

De todos os diagramas da UML, sem dúvida, o diagrama de caso de uso é o mais utilizado. Algumas das vantagens desse diagrama são:

- a) auxilia no levantamento de requisitos (cenários);
- b) pode conter outros documentos que ajudem a entender o negócio;
- c) deve-se trabalhar com versões;
- d) após escrito, peça para outra pessoa ler e explicar o que entendeu. Se houver dúvidas, refaça;
- e) peça ao usuário para assinar os caso de uso aprovados;
- f) deixe toda a documentação publicada na intranet ou extranet.

5.6 Exemplos e exercícios

Uma vídeo locadora deseja cadastrar seu acervo de fitas e CDs, cadastrar os clientes, possibilitar a consulta do acervo, fazer a locação selecionando fita e cliente, realizar a entrega gerando dados para o sistema financeiro.

Solução: Diagrama de contexto – vídeo locadora

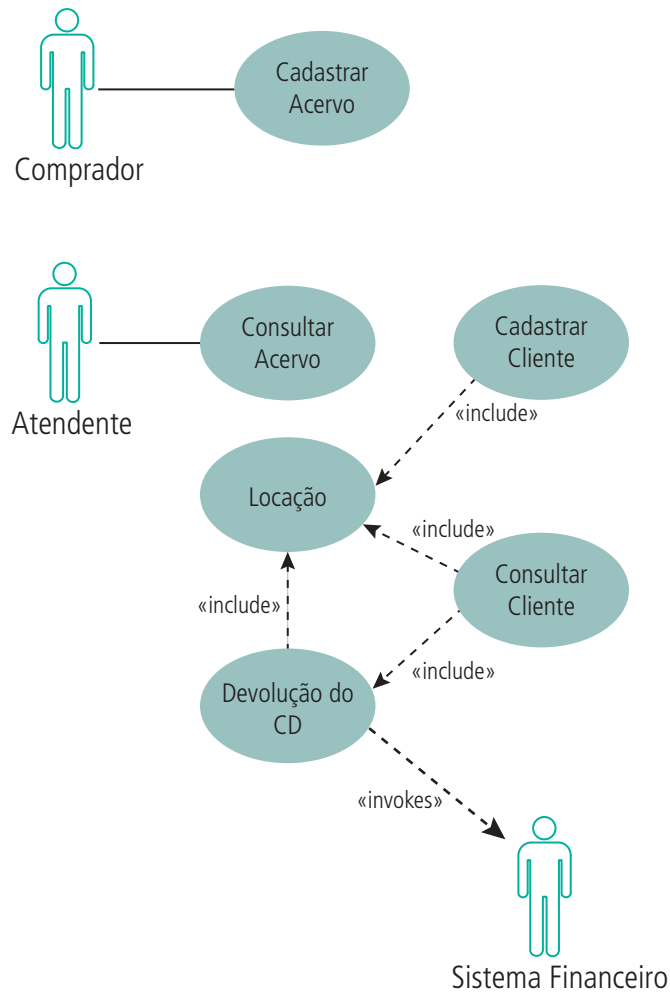


Figura 5.5: Exemplo de diagrama de contexto

Fonte: Elaborada pelo autor Edson Marchetti da Silva (*software Enterprise Architect 7.5*)

Descrição de caso de uso:

Caso de Uso Cadastrar Acervo	
Atores	Comprador
Descrição	Ao receber novas aquisições de fitas e CDs, o comprador deverá cadastrá-las no sistema.
Ação do ator Resposta do sistema	
1. Comprador informa dados do filme	2. Exibe mensagem Cadastramento Ok.

Caso de Uso Cadastrar Cliente	
Atores	Atendente
Descrição	Este caso de uso ocorre para transcrever as fichas cadastrais manuais para o sistema, ou quando se desejar cadastrar um novo cliente no sistema.
Ação do ator Resposta do sistema	
1. Atendente informa dados cadastrais do cliente.	2. Exibe mensagem Cadastramento Ok.

Caso de Uso Consulta acervo	
Atores	Atendente
Descrição	Ocorre quando o atendente desejar consultar o acervo.
Ação do ator Resposta do sistema	
1. O atendente informa (nome, autor, estilo, etc) como sendo filtro de consulta do acervo.	2. Exibe todos os filmes de acordo com o critério informado.

Figura 5.6: Caso de uso – Exemplo 1 – Descrição de caso de uso em forma de ficha

Fonte: Elaborada pelo autor Edson Marchetti da Silva

Caso de Uso Locação	
Atores	Atendente
Descrição	Este caso de uso ocorre quando um cliente desejar fazer locações de filmes disponíveis no acervo.
Ação do ator Resposta do sistema	
1. Este caso de uso começa quando um cliente vai até o Atendente para locar os filmes	
2. Atendente informa o código do cliente (carteira de sócio).	3. Sistema verifica restrições do cliente.
4. Atendente informa o código dos filmes escolhidos pelo cliente.	5. Sistema emite recibo de locação.
Alternativa: cliente não tem carteira de sócio.	
2.1.1 – Executa caso de uso Consulta Cliente	2.1.2. Sistema retorna o código. Retorna para o cenário principal do item 2.
Alternativa: Cliente não está cadastrado.	
2.2.1 – Executa caso de uso Cadastra cliente.	2.2.2. Retorna para o cenário principal do item 2.

Figura 5.7: Caso de uso – Exemplo 2 – Descrição de caso de uso em forma de ficha

Fonte: Elaborada pelo autor Edson Marchetti da Silva

Caso de Uso	Devolução
Atores	Atendente
Descrição	Este caso de uso ocorre quando um cliente devolve parte de todos os filmes locados.
Ação do ator	Resposta do sistema
1. Este caso de uso começa quando um cliente vai até o Atendente para devolver os filmes	
2. Atendente informa o código do cliente (carteira de sócio).	3. Sistema verifica todas as locações pendentes do sistema.
4. Atendente escolhe quais filmes serão devolvidos.	5. Sistema calcula os valores devidos e exhibe na tela.
6. Cliente efetua o pagamento.	7. Sistema emite recibo.
	8. Sistema envia dados para o Sistema financeiro.
Alternativa: Cliente não tem a carteira de sócio.	
1.1. Executa Caso de uso Consulta Cliente	2.1. Sistema retorna o código. Retorna para o cenário principal do item 2.
Alternativa: Cliente após ver total resolve mudar devolução.	
6.1. Atendente informa que cliente que mudar devolução.	6.2. Retorna para o cenário principal do item 4.

Figura 5.8: Caso de uso – Exemplo 3 – Descrição de caso de uso em forma de ficha

Fonte: Elaborada pelo autor Edson Marchetti da Silva

5.7 Diagrama de classes

Uma classe pode ser vista como um modelo a partir do qual os objetos são criados (instanciados). Um objeto é uma instância de uma classe. Podemos imaginar uma classe como um tipo (da mesma forma que em linguagens estruturadas temos o *int*, *float*, ou o *char*). O diagrama de classes descreve os tipos de objetos no sistema e os vários tipos de relacionamentos que existem entre eles.

5.7.1 Representações de uma classe

Nome – deve descrever o que a classe representa ou faz de forma clara;

Atributos – características da classe que devem ser representadas;

Operações – são as funcionalidades da classe.

Regras:

- a) **Nome** – substantivo ou expressão breve – Maiúsculas no primeiro caractere de cada palavra. Ex.: Cliente; ContaCorrente;
- b) **Atributos** – substantivo ou expressão breve que representam alguma propriedade da classe. Ex.: nomCliente; codUfComercial;

- c) **Operações** – verbo, representando algum comportamento da classe. Ex mostrar(); obterSaldo().

Um bom projeto de classe deve:

- a) conter atributos privados;
- b) sempre inicializar os dados;
- c) nem todos os atributos precisam conter métodos *get* e *set*;
- d) organizar classes e distribuir as responsabilidades (funções).

5.7.2 O diagrama de classe

É um dos principais diagramas da UML. Descreve os tipos de objetos e os vários tipos de relacionamentos que existem entre eles. Mostra também atributos e operações de uma classe.

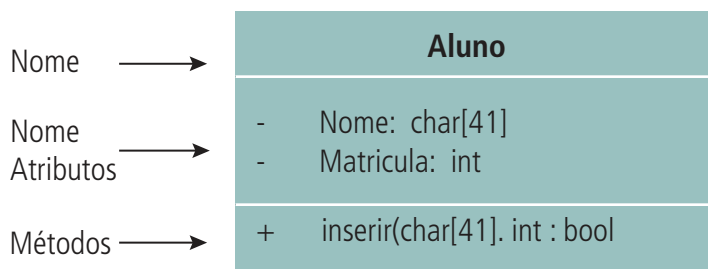


Figura 5.9: Exemplo de diagrama de classe

Fonte: Elaborada pelo autor Edson Marchetti da Silva

5.8 Relacionamentos entre classes

Os objetos têm relações entre si; ex.: um professor **ministra** uma disciplina **para** estudantes **numa** sala. Estas relações são conhecidas na UML como:

- a) Associação/Agregação e Composição;
- b) Dependência;
- c) Generalização (herança).

5.8.1 Associação

São relacionamentos entre duas classes representados por uma linha sólida. Especificam que objetos de uma classe estão ligados a objetos de outras classes. Podemos ter associação unária, binária etc.

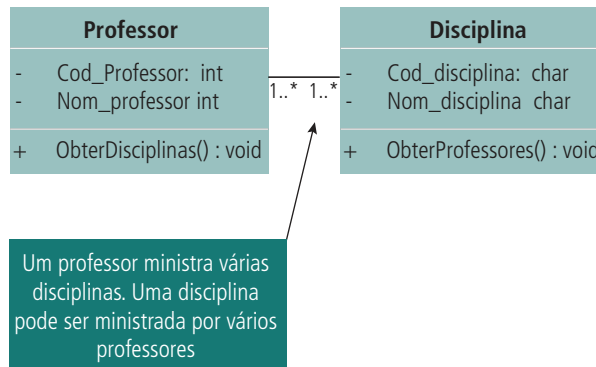
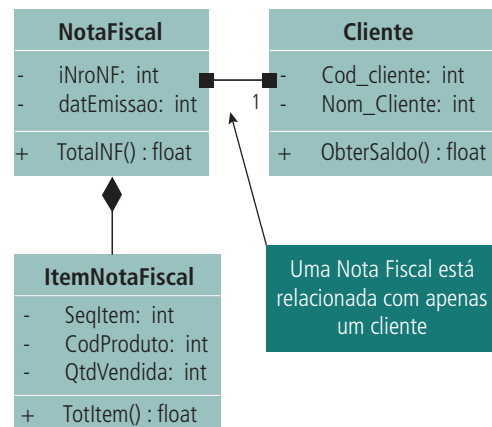


Figura 5.10: Exemplo de associações

Fonte: Adaptada de Medeiros (2004)

a) Associação unária

Quando há um relacionamento de uma classe para ela mesma.

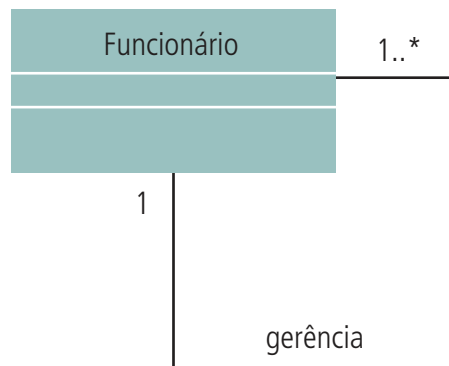


Figura 5.11: Exemplo de associação unária

Fonte: Elaborada pelo autor Edson Marchetti da Silva

b) Associação binária

Quando há duas classes envolvidas na forma direta de uma para a outra.

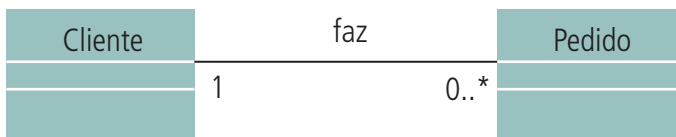


Figura 5.12: Exemplo de associação binária

Fonte: Elaborada pelo autor Edson Marchetti da Silva

c) Associação n-ária

Quando há um relacionamento entre mais de duas classes.

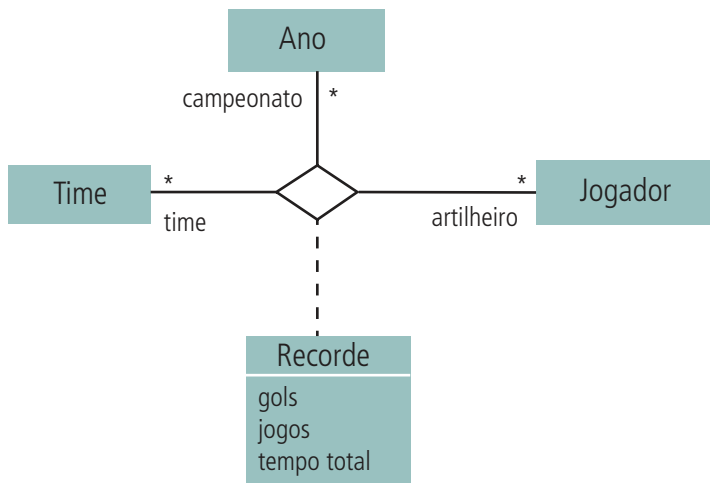


Figura 5.13: Exemplo de associação n-ária

Fonte: Elaborada pelo autor Edson Marchetti da Silva

d) Classe associativa

É introduzida quando uma associação tem propriedades associadas.

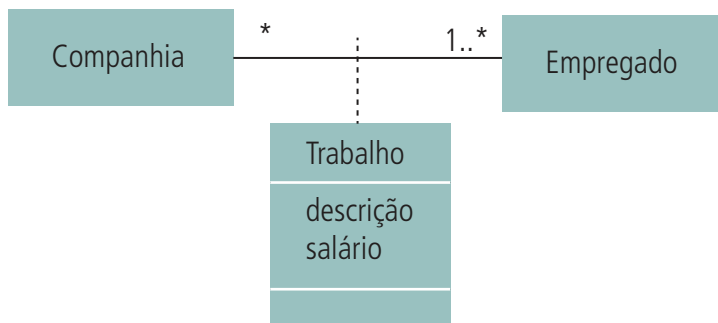


Figura 5.14: Exemplo de classe associativa.

Fonte: Elaborada pelo autor Edson Marchetti da Silva

e) Papéis de associações

Papéis normalmente são colocados nas extremidades de uma associação quando:

- a associação relaciona dois objetos da mesma classe;
- há mais de uma associação entre as classes.

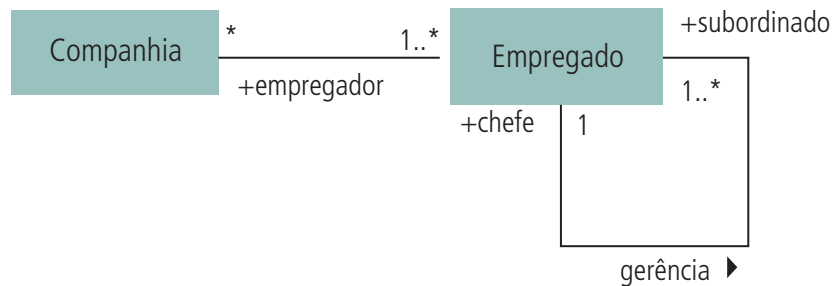


Figura 5.15: Exemplo de papéis

Fonte: Elaborada pelo autor Edson Marchetti da Silva

f) Multiplicidade de associações

Indica quantos objetos podem participar de um dado relacionamento, conforme a convenção a seguir:

"*" - Representa uma variação de zero a infinito.

"1" - Indica relacionamento obrigatório único.

"0..1" - Pode ser nenhum ou um (relacionamento único não obrigatório).

"1..5" - Entre 1 e 5.

"2,4" - Dois ou quatro.

g) Agregação

Tipo de associação (é parte de, todo/parte) em que o objeto parte é um atributo do todo. A agregação faz sentido quando duas classes associadas têm um sentido próprio e quando separadas continuam existindo como unidades autônomas.

Ex.: Uma garagem está associada a um apartamento.

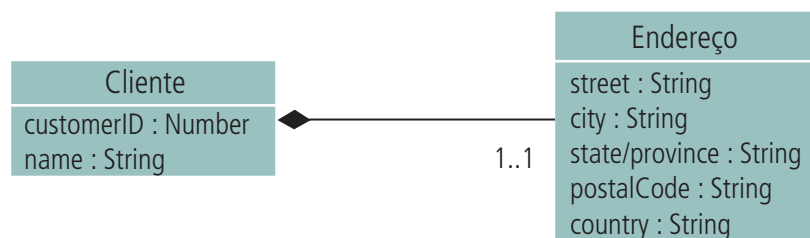


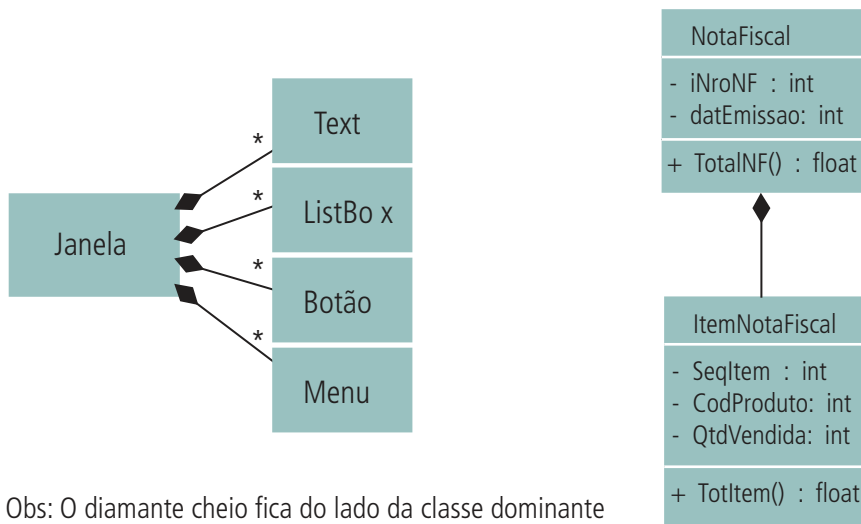
Figura 5.16: Exemplo de agregação

Fonte: Elaborada pelo autor Edson Marchetti da Silva

h) Composição

Relacionamento entre um elemento (o todo) e outros elementos (as partes).

Ex.: Notas fiscais e itens de nota fiscais.



Obs: O diamante cheio fica do lado da classe dominante

Figura 5.17: Exemplo de composição

Fonte: Elaborada pelo autor Edson Marchetti da Silva

i) Dependência

Uma mudança em um elemento pode alterar a especificação de outro elemento dependente. Essa dependência indica que objetos de uma classe podem utilizar serviços de objetos de outra classe.

j) Generalização (herança)

Relacionamento entre uma classe mais geral e outra mais específica. A classe mais específica herda os atributos e métodos da classe mais geral.

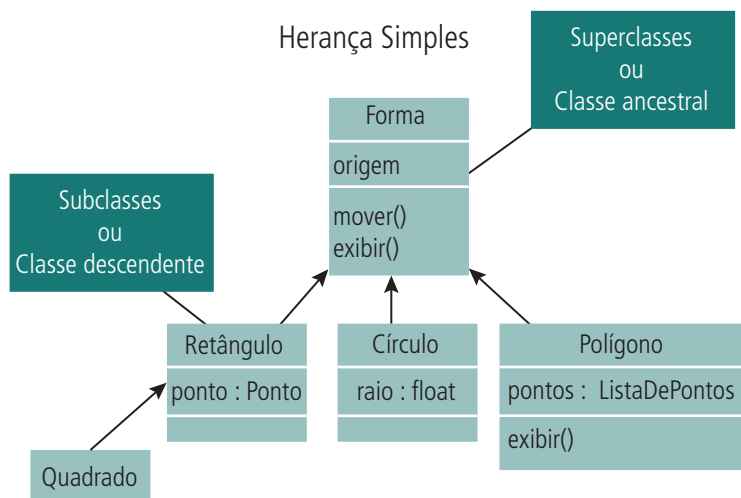


Figura 5.18: Exemplo de herança simples

Fonte: Elaborada pelo autor Edson Marchetti da Silva

5.9 Exemplo de um diagrama de classe

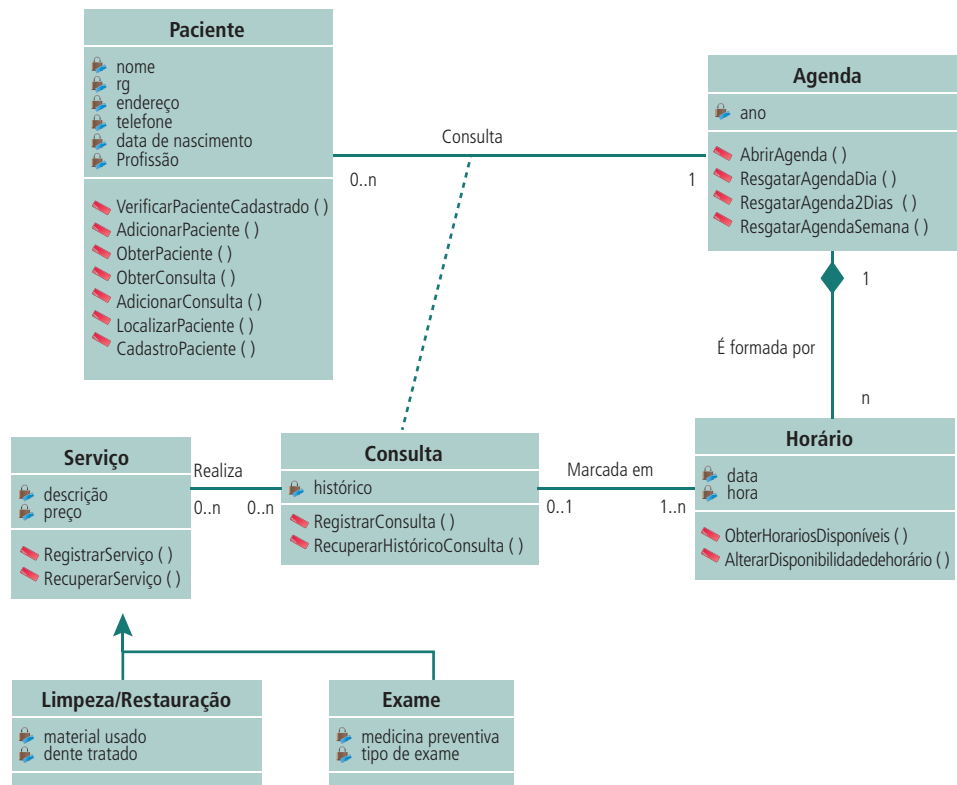


Figura 5.19: Exemplo de diagrama de classe

Fonte: http://www.macoratti.net/net_uml6.gif

Resumo

Caro estudante, nesta penúltima aula foram estudados dois dos principais diagramas da UML: Diagrama de casos de uso e Diagrama de classes. Foi descrito como criar e analisar esses diagramas no Projeto de Sistemas.

Atividades de aprendizagem

1. O reitor de uma universidade privada deseja implantar um sistema de informação na biblioteca a fim de controlar os empréstimos de livros para a comunidade acadêmica (estudantes, funcionários e professores). Você foi convidado a fazer a especificação dos requisitos desse sistema.

O sistema deverá possibilitar que:

- a bibliotecária cadastre as tabelas básicas (autores, editoras e áreas do conhecimento), cadastre livros e cadastre usuários;
- os usuários consultem o acervo;
- o atendente possa realizar o empréstimo e, ao fazê-lo, tenha acesso a consultar o acervo e cadastrar usuário caso ele não exista. Deve-se verificar também se existem pendências para liberar ou bloquear o empréstimo. O atendente será responsável por informar dados da devolução do(s) livro(s) e, nesse ato, será verificado se houve atraso na entrega e, se houve, será gerado automaticamente o débito para o usuário no Sistema de Contas a Pagar para incluir o lançamento na próxima boleto.

Tomando como base as informações acima, levantadas a partir de uma entrevista com os membros da Universidade, construa a especificação do caso de uso para os eventos emprestar livros e devolver livros. Monte os quadros conforme modelo apresentado a seguir.

Caso de uso			
Atores			
Descrição			
Ação do ator		Resposta do sistema	
Nro			

2. Gerar um Diagrama de classe para modelar o seguinte problema:

Deseja-se informatizar o registro escolar de uma escola. Sabe-se que a escola possui estudantes e que devem ser mantidas informações da matrícula, nome, endereço e curso. Os estudantes são matriculados em turmas. Você deverá mostrar dados completos de um dado estudante e mostrar todos os estudantes de uma dada turma.

3. Marque a alternativa correta:

- a)** O diagrama de sequência mostra uma interação, isto é, uma sequência de mensagens trocadas entre vários objetos num determinado contexto; enfatiza a comunicação e passagem de controle entre objetos ao longo do tempo.
- b)** O diagrama de caso de uso é um diagrama comportamental que descreve uma interação entre um usuário e um sistema, ou seja, um cenário (normal e os alternativos).
- c)** O diagrama de classe está relacionado ao modelo de solução do problema a partir de uma visão dinâmica do sistema em execução.
- d)** O diagrama de transição de estado mostra os diferentes estados de um objeto durante sua vida, e o estímulo que faz com que o objeto mude seu estado (através da chamada de um método de uma classe).
- e)** O diagrama de atividade descreve uma sequência de atividades com suporte para comportamento condicional e paralelo.

Registre suas respostas num arquivo e poste-o no AVEA.

Aula 6 – Demais diagramas da UML

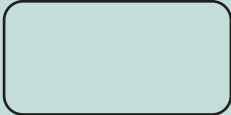
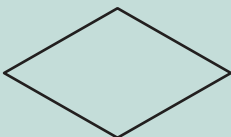
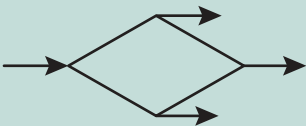
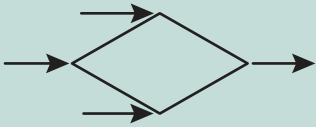
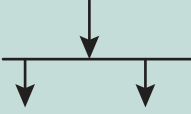
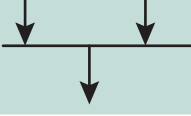
Objetivos

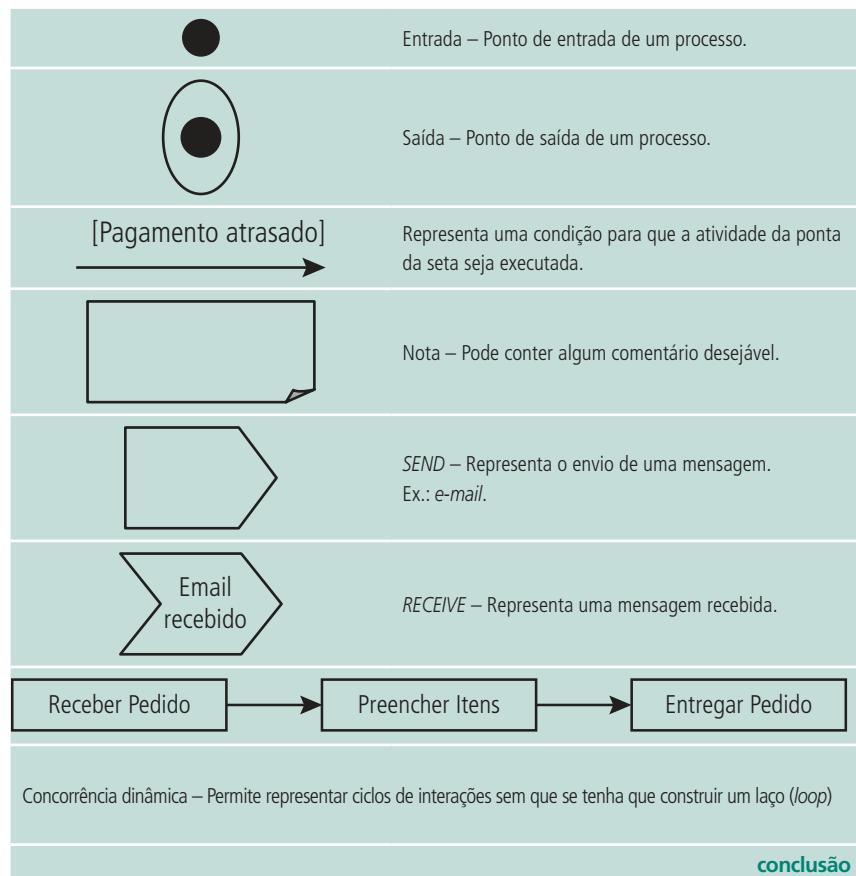
Criar e analisar os diagramas apresentados.

6.1 Diagrama de atividade

O diagrama de atividade tem como objeto modelar um conjunto de atividades no sistema. Essas atividades podem ser executadas através do controle de condições ou até mesmo em paralelo. O diagrama de atividade é utilizado também para modelar graficamente (*threads*) em programas concorrentes.

6.1.1 Principais componentes

	Representa uma atividade: pode ser uma ação do usuário ou uma resposta do sistema.
	Representa a passagem de uma atividade para outra.
	Condicional – Representa um símbolo de decisão.
	Desvios – Possui uma transição de entrada única e várias transições de saída mutuamente exclusivas.
	Intercalações – múltiplas transições de entrada mutuamente exclusivas e uma saída
	Fork – Representa que a partir deste ponto ocorrerão atividades que serão processadas
	Join – Representa que mais de uma atividade chegou neste ponto. Somente após todas chegarem é que prossegue para a próxima
continua	



Exemplo 1 – Utilização do diagrama de atividade:

O departamento de vendas de uma empresa de representação recebe os pedidos de seus vendedores externos. Após avaliar o crédito do cliente, emite a nota fiscal enviando a 1ª via para o departamento de expedição e a 2ª para o departamento financeiro. Enquanto a expedição separa as mercadorias disponibilizando para fazer a entrega, o financeiro gera a cobrança bancária e aguarda receber o pagamento. Ao fim desse processo, o pedido poderá ser finalizado. Para os cliente inadimplentes será enviada uma cobrança ficando o pedido cancelado.

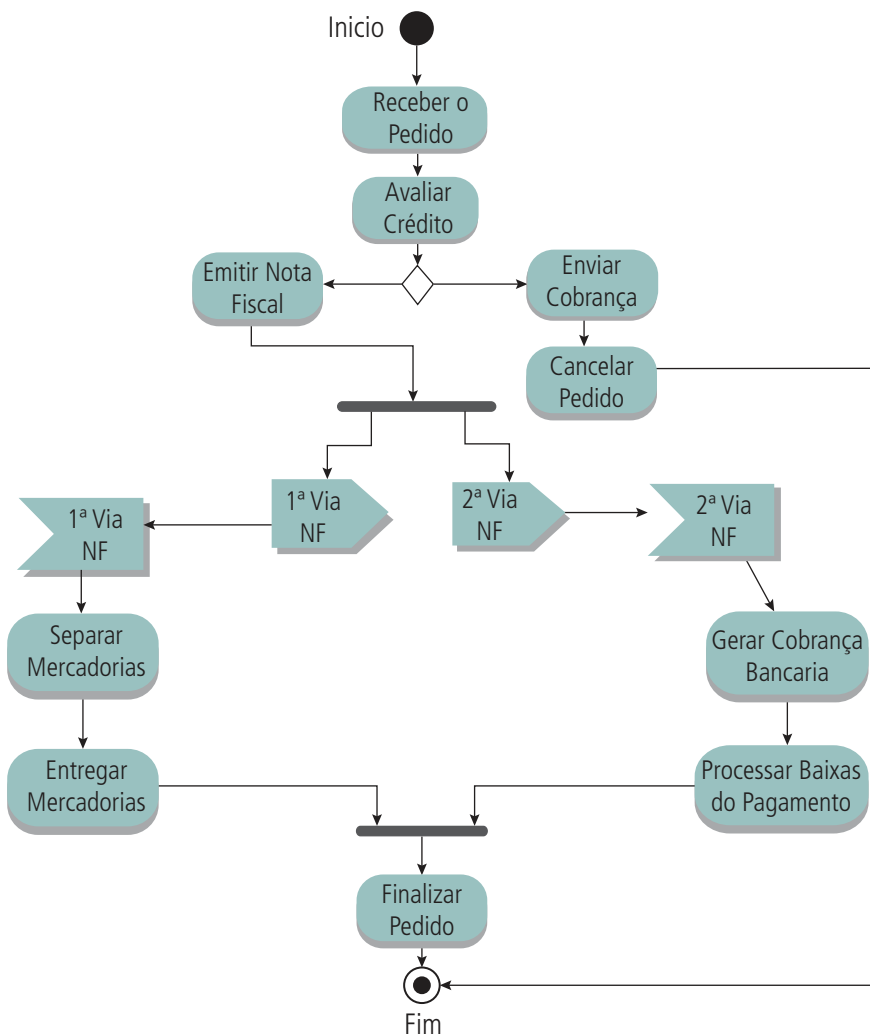


Figura 6.1: Solução do Exemplo 1 com swimLines

Fonte: Elaborada pelo autor Edson Marchetti da Silva

6.2 Diagrama de sequência

Mostra visualmente a sequência de ações que o *software* tomará diante de uma interação com um ator.

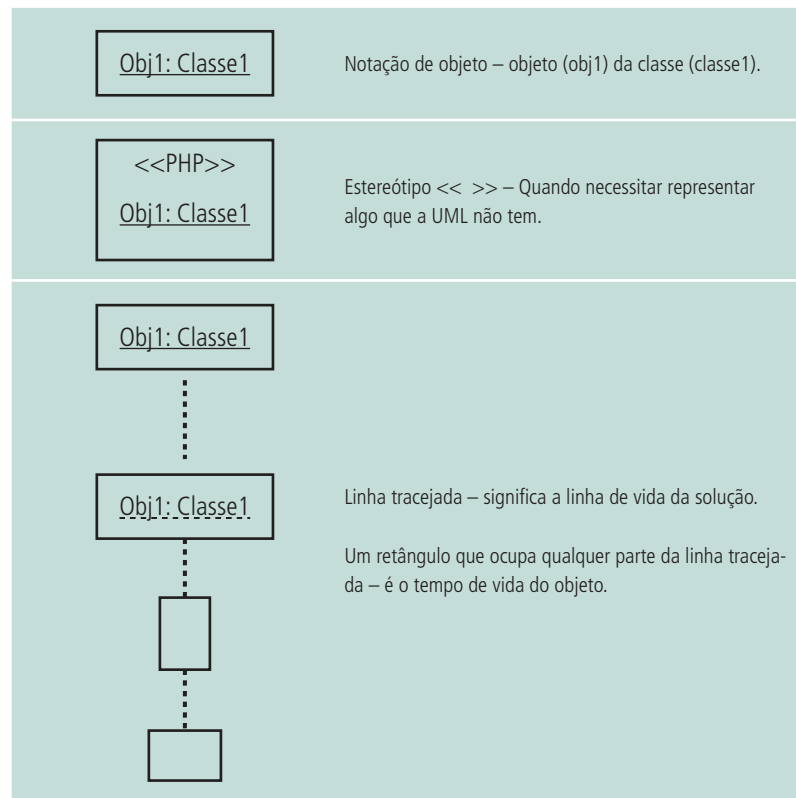
Baseia-se no diagrama de casos de uso.

1 caso de uso → N Diagramas de Sequência

Baseia-se, também, no diagrama de classes.

Fornece as classes e os métodos associados.

Notação:



6.2.1 Mensagem síncrona

São mensagens enviadas das quais se espera um retorno para prosseguir.

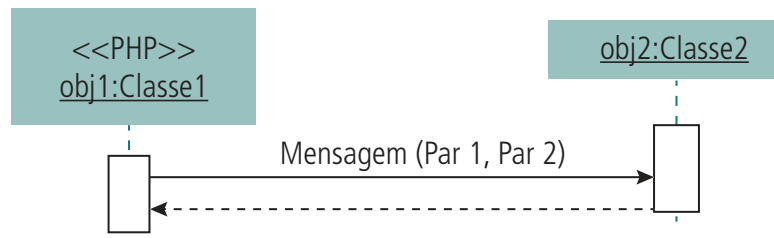


Figura 6.2: Exemplo de mensagem

Fonte: Elaborada pelo autor Edson Marchetti da Silva

- a) **Seta** – objeto enviando uma mensagem para outro objeto;
- b) **Seta tracejada** – representa o retorno da mensagem.

6.2.2 Mensagem assíncrona

São mensagens enviadas, das quais, não se espera um retorno imediato.

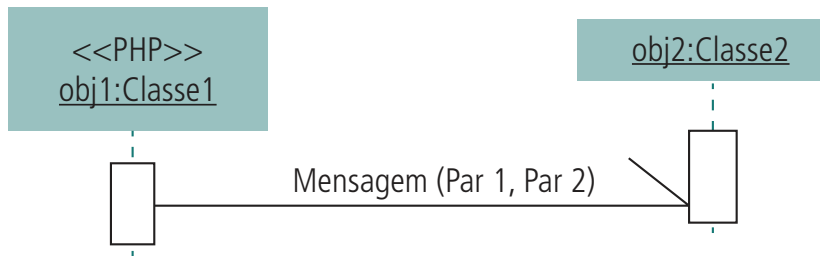


Figura 6.3: Exemplo de mensagem assíncrona.

Fonte: Elaborada pelo autor Edson Marchetti da Silva

a) Seta com um traço – objeto enviando uma mensagem para outro objeto.

Ex.: Impressão de um relatório.

b) Mensagem recursiva – representa uma mensagem que é enviada para o próprio objeto.

Ex.: uma classe que recebe uma senha; chame seu método de criptografia antes de enviar a senha para validação.

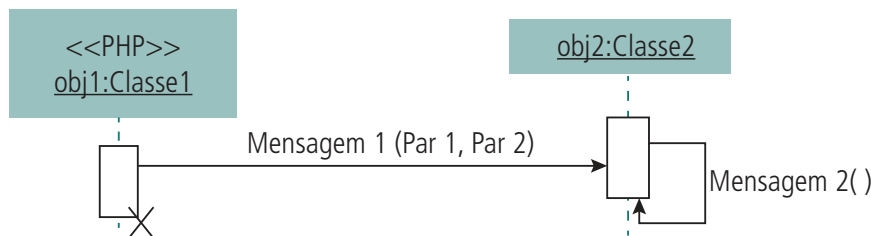


Figura 6.4: Exemplo de mensagem recursiva

Fonte: Elaborada pelo autor Edson Marchetti da Silva

O "X", no final da ocorrência de execução indica que o objeto irá se autodestruir.

Exemplo de diagrama de sequência:

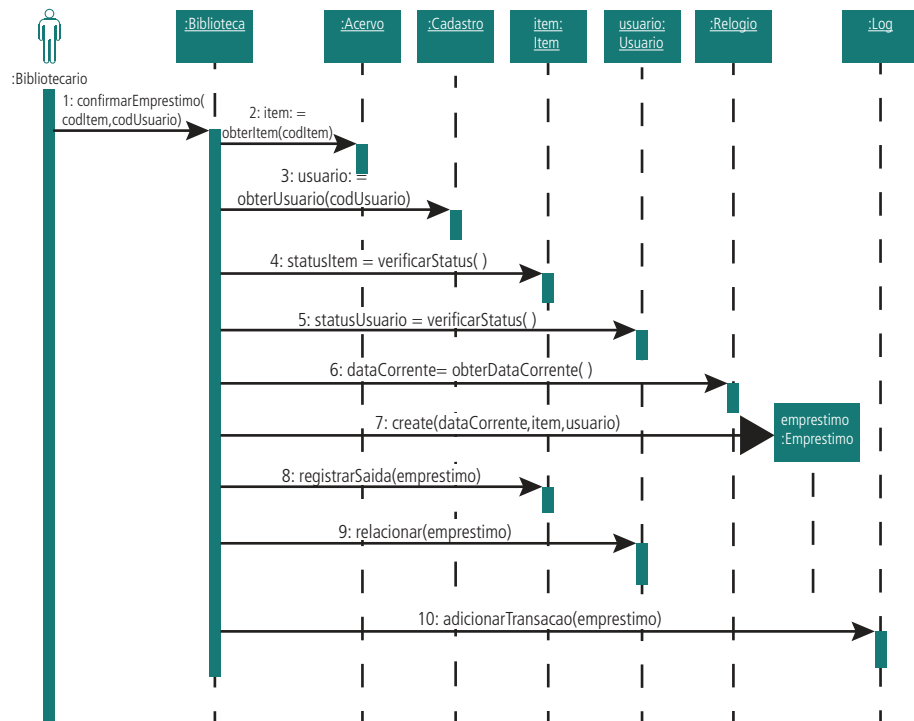


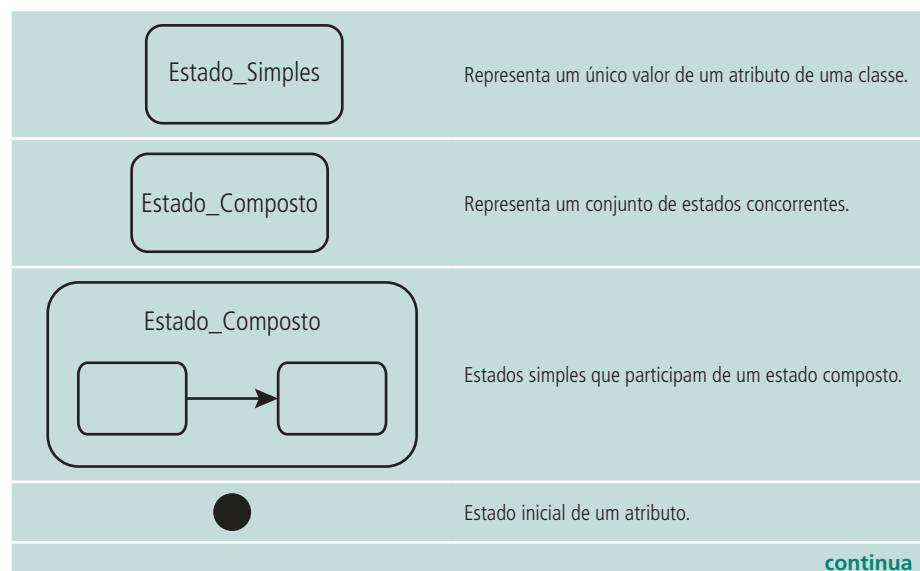
Figura 6.5: Exemplo de diagrama de sequência

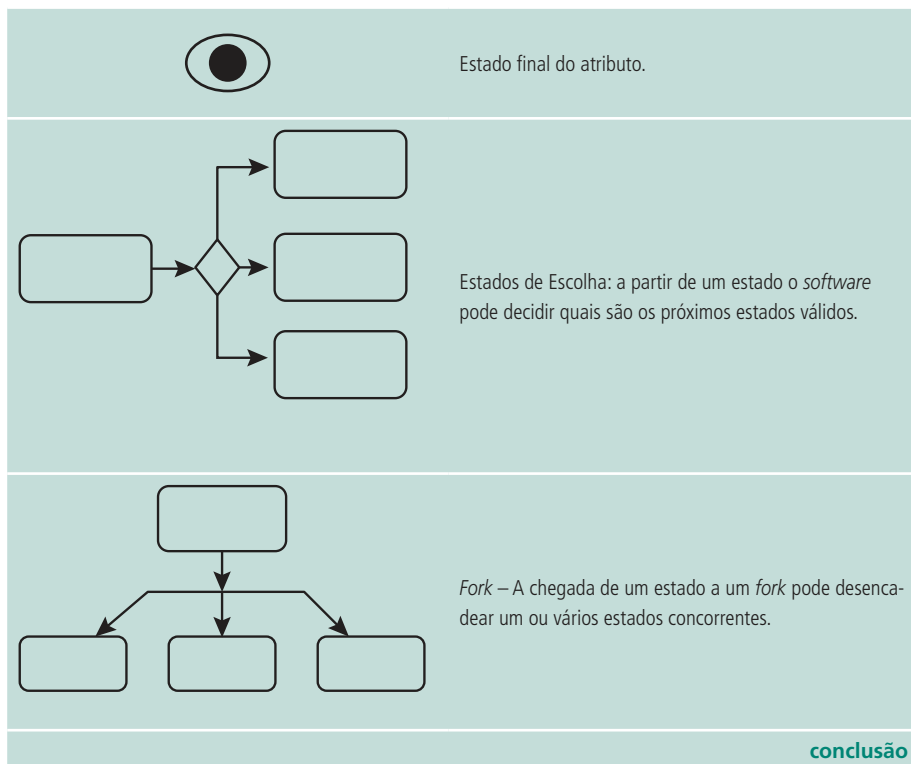
Fonte: <http://www.unicamp.br/~hans/mc426/confirmarEmprestimoSeq.jpg>

6.3 Diagrama de transição de estado

Mostra os diferentes estados de um objeto durante sua execução e o estímulo que faz com que o objeto mude de estado.

Notação:





Veja na Figura 6.6 um exemplo de diagrama de transição de estado.

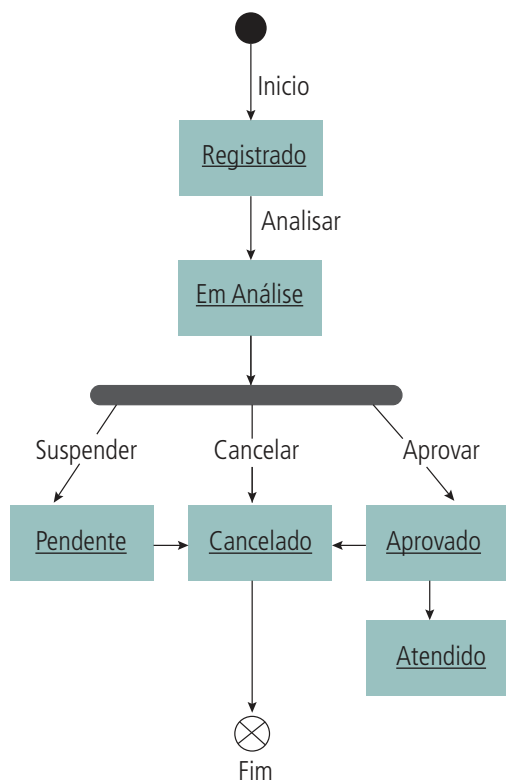


Figura 6.6: Exemplo de diagrama de transição de estado

Fonte: Elaborada pelo autor Edson Marchetti da Silva

6.4 Diagrama de pacotes

Um pacote pode conter qualquer diagrama UML. Os mais comuns são:

- a) Diagrama de classe – ajuda no agrupamento das classes afins;
- b) Diagrama de caso de uso – ajuda na abstração do domínio do problema.

Como escolher as classes que pertencem a um pacote?

- a) Classes de uma mesma hierarquia de herança;
- b) Classes que estejam ligadas por agregação ou composição;
- c) Classes que estejam em um mesmo diagrama de sequência e que tenham muitas colaborações (alto acoplamento).

Veja nas Figuras 6.7 e 6.8 alguns exemplos de diagrama.

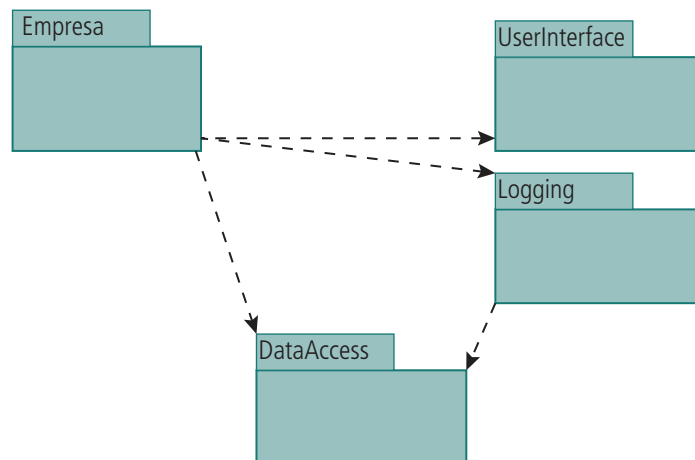


Figura 6.7: Exemplo de diagrama de pacotes

Fonte: <http://www.devmedia.com.br/a-importancia-do-design-na-melhoria-da-eficacia-do-desenvolvedor-em-projetos-de-construcao-de-software/11806>

6.4.1 Utilização do diagrama de pacotes em caso de uso

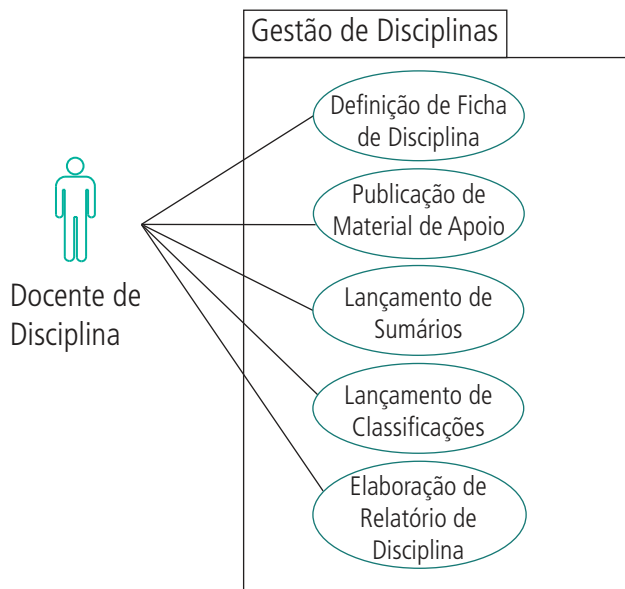


Figura 6.8: Exemplo de diagrama de pacotes com caso de uso

Fonte: Elaborada pelo autor Edson Marchetti da Silva

6.5 Diagrama de componentes e implantação

Os diagramas de componentes mostram os elementos reutilizáveis de *software* e sua interdependência. Um componente é qualquer arquivo que contenha uma parte necessária para o funcionamento do *software*. Ex.: páginas HTML, arquivos de configuração **.ini**, arquivos executáveis **.exe**, **.dll**, **.jar** etc.

a) Componente: descreve um módulo físico do sistema.

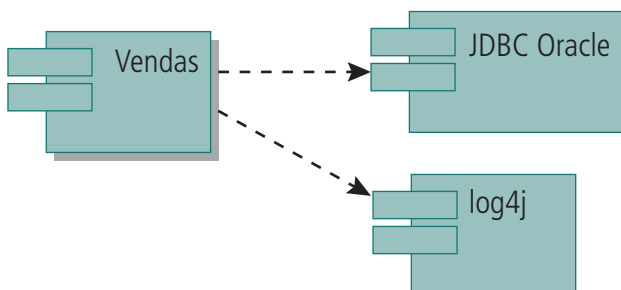


Figura 6.9: Exemplo de diagrama de componente.

Fonte: <http://www.devmedia.com.br/uml-unified-modeling-language-parte-03/9505>

- b) Interface:** é uma coleção de operações usadas para especificar um serviço disponibilizado pelo componente.

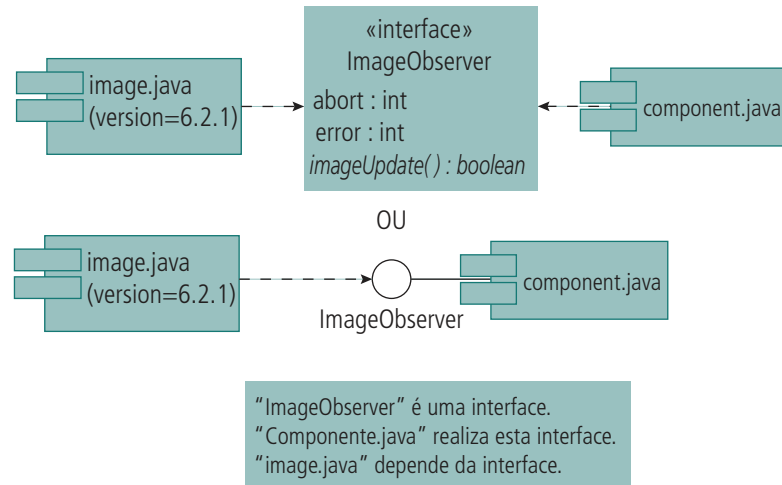


Figura 6.10: Exemplo de aplicação

Fonte: Elaborada pelo autor Edson Marchetti da Silva

- c) Dependência:** quando depende de outro componente ou de uma interface.

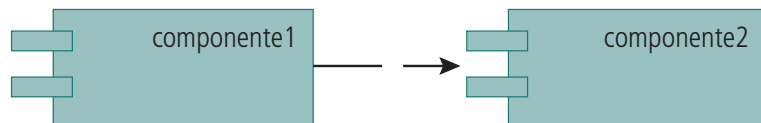


Figura 6.11: Exemplo de dependência

Fonte: Elaborada pelo autor Edson Marchetti da Silva

- d) Realização:** quando implementa uma interface.

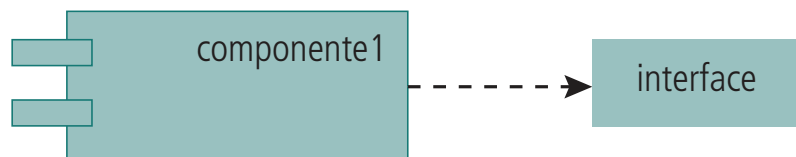


Figura 6.12: Exemplo de realização

Fonte: Elaborada pelo autor Edson Marchetti da Silva

e) Generalização: quando é uma especialização de outro componente.

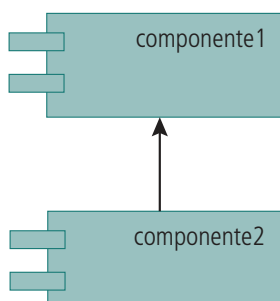


Figura 6.13: Exemplo de realização

Fonte: Elaborada pelo autor Edson Marchetti da Silva

f) Associação: quando é associado a outro componente.

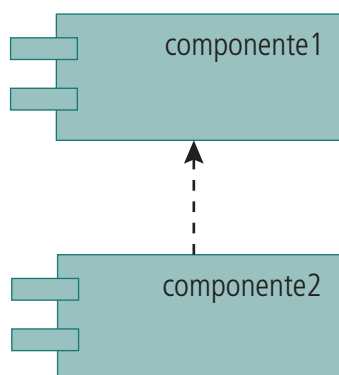


Figura 6.14: Exemplo de associação

Fonte: Elaborada pelo autor Edson Marchetti da Silva

6.6 Diagrama de implantação

Fornece uma maneira de visualizar a relação física entre os componentes de *software* e *hardware*. Esse diagrama mostra os nodos do sistema, os componentes existentes e suas conexões:

- a)** um nodo representa uma parte do *hardware* (servidor, uma estação cliente). Dentro de um nodo podem existir um ou mais componentes;
- b)** uma conexão representa um caminho de comunicação entre nodos;
- c)** um nodo é um objeto físico que representa um recurso. Esse recurso pode ser um dispositivo com memória e capacidade de processamento.

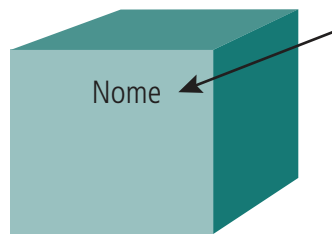


Figura 6.15: Exemplo de nodo

Fonte: Elaborada pelo autor Edson Marchetti da Silva

Uma conexão representa a ligação de *hardware* entre dois nodos.

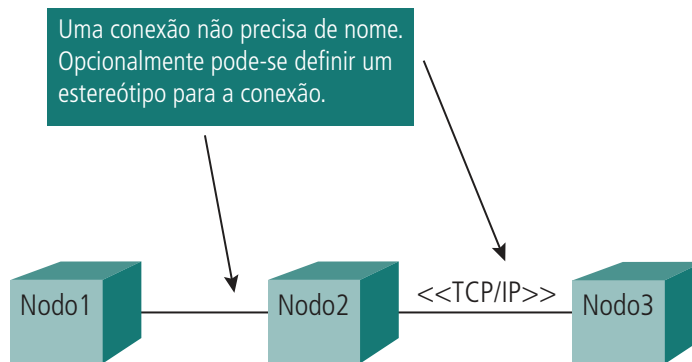


Figura 6.16: Exemplo de conexão

Fonte: Elaborada pelo autor Edson Marchetti da Silva

A conexão entre o *software* e o *hardware* é definida pelo relacionamento entre nodos e componentes. Um nodo executa um ou mais componentes.

Resumo

Prezado estudante, nesta última aula vimos outros seis diagramas da UML: Diagrama de atividade, sequência, transição de estados, bem como os diagramas de pacotes, componentes e instalação.

Atividades de aprendizagem

1. Desenhe um diagrama de atividades mostrando as *swimlanes* que representem a situação descrita a seguir de um ambulatório hospitalar.

O paciente ao chegar ao hospital primeiramente se encaminha à recepção. Na recepção o paciente apresenta os seus documentos identificando o convênio que será utilizado, em seguida ele informa seus sintomas. A recepcionista, juntamente com o paciente, define a especialidade médica e a urgência para o atendimento; dessa forma, o paciente entra numa fila automatizada de espera. A secretária que controla as consultas verifica a cada disponibilidade do médico o próximo da fila a ser chamado. Quan-

do o paciente é atendido na consulta, o médico registra a anamnese, os dados do exame clínico, do diagnóstico e cadastra os medicamentos ou os exames se forem necessários. O paciente retorna à secretaria, que o encaminha para a farmácia, ou para o laboratório, se houver exames a ser feitos. O laboratório coleta material e registra no sistema o resultado dos exames. Quando o paciente retorna à secretaria, ele é encaminhado para o médico assim que houver uma disponibilidade. Após o médico avaliar os exames via intranet, o paciente é encaminhado para internação ou simplesmente para a farmácia. Obs.: os exames ficam disponíveis via extranet para o paciente.

2. De todos os diagramas estudados até o momento, cite dois que você considera mais importantes e justifique a sua resposta.
3. Descreva a finalidade de :
 - - Diagrama de pacote;
 - - Diagrama de componente;
 - - Diagrama de implantação.
4. Desenhe um diagrama de atividades com raias, para representar a situação descrita de uma empresa que vende computadores diretamente na loja ou através de entrega para todo o Brasil.

O departamento de vendas recebe o pedido de compra de cliente, na loja ou via *telemarketing*, e encaminha simultaneamente uma cópia para a expedição separar os itens e outra para o setor financeiro avaliar o crédito do cliente. Caso não haja estoque ou o crédito não for aprovado, o pedido é finalizado. Caso contrário Vendas emite a nota fiscal e dispara simultaneamente uma via da NF para a expedição processar a entrega avaliando: se o cliente estiver na loja, entrega a mercadoria; se não, encaminha para a transportadora. A outra via da NF vai para o setor financeiro, que providencia a emissão das boletas e controla o recebimento.

Registre suas respostas num arquivo e poste-o no AVEA.

Referências

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML: guia do usuário**. 7ª. reimp. Rio de Janeiro: Elsevier, 2005.

PAULA FILHO, Wilson de Pádua. **Engenharia de software: fundamentos métodos e padrões**. 2ª ed. Rio de Janeiro: LTC, 2003.

LARMAN, Craig. **Utilizando UML e padrões**. 3ª ed. Porto Alegre: Bookman, 2007.

MCMENAMIM, Sthephen M.; PALMER John F. **Análise essencial de sistemas**. São Paulo: McGraw-Hill, 1991.

MEDEIROS, Ernani S. **Desenvolvendo software com UML 2.0: definitivo**. São Paulo: Pearson Makron Books, 2004.

POMPILHO, S. **Análise essencial: guia prático de análise de sistemas**. Rio de Janeiro: Infobook, 2002.

YORDON, Edward. **Análise estruturada moderna**. Rio de Janeiro: Campus, 1990.

Currículo dos professores-autores

Gustavo Campos Menezes é graduado em Tecnologia em Informática pelo Unicentro Newton Paiva e mestre em Ciência da Computação pela UFMG. Professor efetivo do CEFET-MG desde setembro de 2006, trabalha principalmente com disciplinas relacionadas a Linguagens de Programação e Algoritmos para cursos de Informática (Ensino técnico) e Engenharias (Graduação).



Edson Marchetti da Silva é professor corresponsável pela elaboração do material da disciplina de Projetos de Sistemas do curso de PGTI. Atua como professor no CEFET MG desde 2007, lecionando nos cursos Técnicos em Informática, Técnico em Eletromecânica, na graduação em Engenharia Mecatrônica e na pós graduação *Lato Sensu*. Ministra disciplinas relacionadas a Banco de Dados, Projeto de Sistemas, Lógica e Programação de Computadores. Já trabalhou como professor na Universidade Federal de Ouro Preto, no Curso de Graduação em Sistemas de Informação entre os anos de 2005 e 2007. Na iniciativa privada atuou por 23 anos em diversos cargos, desde programador de computadores, analista de sistemas, administrador de banco de dados e gerente de informática. É graduado em Tecnologia de Processamento de Dados, 1991, em Ciência da Computação, 2003, especialista em Engenharia de *Software* pelo Instituto de Educação Continuada da Universidade Católica de Minas Gerais, 2001, mestre em Administração pela Universidade FUMEC, 2006, e doutorando em Ciências da Informação pela Universidade Federal de Minas Gerais. Atualmente, atua como professor nos cursos Técnico em Informática e como coordenador adjunto do Curso de Pós-Graduação *lato sensu* em Banco de Dados, além de atuar no ensino de educação a distância (EAD).





ISBN 978-85-99872-25-3

